



SCUOLA POLITECNICA E DELLE SCIENZE DI BASE
DIPARTIMENTO DI INGEGNERIA INDUSTRIALE

CLASSE DELLE LAUREE MAGISTRALI IN INGEGNERIA AEROSPAZIALE E
ASTRONAUTICA (LM 20)

COURSE
FLUIDODINAMICA NUMERICA

Project Work

A study on two-dimensional wake flows: fluid simulation of a Kármán vortex street past a circular cylinder by computing *streaklines* and comparison with a four-digit NACA airfoil

SUPERVISOR:
PROF. GENNARO COPPOLA

STUDENT:
MATTEO DE LUCA PICIONE
ID: M53001994

ACADEMIC YEAR 2024 – 2025

COMPUTATIONAL FLUID DYNAMICS

A study on two-dimensional wake flows: fluid simulation of a Kármán vortex street past a circular cylinder by computing *streaklines* and comparison with a four-digit NACA airfoil

Using *CFD* techniques, the study employs the ψ - ζ model to solve the incompressible Navier-Stokes equations and track *streaklines*, allowing for qualitative analyses of wake structures. Simulations are performed on a collocated grid with appropriate boundary conditions, ensuring stability and accuracy through a finite difference approach and an explicit Runge-Kutta method. Results highlight the distinct wake behavior of each geometry, with the circular cylinder generating a well-defined vortex street, whereas the airfoil exhibits reduced wake turbulence. The findings demonstrates the impact of body shape on vortex shedding phenomena.

Copyright © 2024–2025
Matteo De Luca Picione
Some rights reserved
Università degli Studi di
Napoli Federico II

(Legge italiana sul Copy-
right 22.04.1941 n. 633)

Contents

1	<i>Psi-Zita</i> model	1
1.1	Vorticity	2
1.2	Stream function	2
1.3	Flow model	3
2	Lines of flow	3
2.1	Streamlines, pathlines and streaklines	3
3	Flow simulation	4
3.1	Domain	5
3.2	Boundary conditions	6
3.3	Discretization of diffusive and convective terms	7
3.4	Time integration	9
3.5	Implementation	10
4	Results	18
	References	30

1 *Psi-Zita* model

Incompressible Navier-Stokes equations (*INS*) are the starting point to obtain the *Psi-Zita* model (ψ - ζ), which is a flow model particularly suited for two-dimensional incom-

pressible fluid flow simulations.

INS are the mass conservation equation and the momentum balance equation. They constitute the set of partial differential equations (1), where (i) $\rho = \rho(\mathbf{x}, t)$ is the density field, (ii) $\mathbf{u} = \mathbf{u}(\mathbf{x}, t) = [u, v, w]^T$ is the

velocity vector field, (iii) $p = p(x, t)$ is the pressure field, (iv) μ is the dynamic viscosity of the fluid, and (v) $f = f(x, t)$ is the body force vector field.

$$\begin{cases} \nabla \cdot \mathbf{u} = 0 \\ \rho \frac{\partial \mathbf{u}}{\partial t} + \rho \mathbf{u} \cdot \nabla \mathbf{u} + \nabla p = \mu \nabla^2 \mathbf{u} + \rho \mathbf{f} \end{cases} \quad (1)$$

In most cases the body force vector field is conservative, thereby it can be rewritten in terms of the gradient of its scalar potential ξ as

$$\mathbf{f} = -\nabla \xi \quad (2)$$

and the *INS* become

$$\begin{cases} \nabla \cdot \mathbf{u} = 0 \\ \frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} + \nabla P = \nu \nabla^2 \mathbf{u} \end{cases} \quad (3)$$

where ν is the kinematics viscosity of the fluid and $P = p + \xi/\rho$ is the generalized pressure field.

Applying the divergence operator at both sides of the momentum balance equation yields

$$\nabla^2 P = -\nabla \cdot (\mathbf{u} \cdot \nabla \mathbf{u}) \quad (4)$$

that is an elliptic equation in the generalized pressure field known as the pressure evolution equation. It shows how pressure and velocity field affect each other, instantaneously adapting at every instant. This behavior is caused by the speed of sound being infinite, as assumed in the *INS*.

A further investigation of equation (4) shows that with this model the pressure field does not depend on the fluid temperature field T , but only on the kinematics of the flow. This mathematically translates to the energy balance equation being unrelated from the set of equations (3). This allow to exclude the energy equation from the *INS*, since they already constitute a well-posed differential problem in the unknowns $\mathbf{u}$ and p .

Furthermore, the pressure in this model does not have a physical meaning, since it depends on the velocity field and its initial condition cannot be chosen arbitrarily. As a result, it cannot be assumed any fluid thermodynamical model, e.g. ideal gas law $p = \rho RT$ must not be considered.

1.1 Vorticity

The flow vorticity is defined as

$$\boldsymbol{\omega} = \nabla \times \mathbf{u} \quad (5)$$

and it allows to rewrite the convective term in the momentum balance equation as follows

$$\mathbf{u} \cdot \nabla \mathbf{u} = \boldsymbol{\omega} \times \mathbf{u} + \nabla \left(\frac{|\mathbf{u}|^2}{2} \right) \quad (6)$$

Substituting equation (6) into the momentum balance equation and applying the curl operator to both of its sides gives the following

$$\frac{\partial \boldsymbol{\omega}}{\partial t} + \mathbf{u} \cdot \nabla \boldsymbol{\omega} - \boldsymbol{\omega} \cdot \nabla \mathbf{u} = \nu \nabla^2 \boldsymbol{\omega} \quad (7)$$

known as the vorticity equation.

It is worth noticing how this equation can be simplified when analyzing a two-dimensional flow. In particular, in this case the vorticity is

$$\boldsymbol{\omega} = \nabla \times \mathbf{u} = \left(\frac{\partial v}{\partial x} - \frac{\partial u}{\partial y} \right) \mathbf{k} = \omega \mathbf{k} \quad (8)$$

with $\mathbf{k}$ being the z-axis versor orthogonally to the motion plane xy and ω the projection of $\boldsymbol{\omega}$ along that direction. As a result, equation (7) now becomes

$$\frac{\partial \omega}{\partial t} + \mathbf{u} \cdot \nabla \omega = \nu \nabla^2 \omega \quad (9)$$

1.2 Stream function

In the case of a non-compressible fluid flow, the velocity vector field is non-divergent, thereby there exists a vector field $\boldsymbol{\psi}$ called vector potential, such that

$$\mathbf{u} = \nabla \times \boldsymbol{\psi} \quad (10)$$

It can easily be verified that when the flow is two-dimensional, the vector potential is $\boldsymbol{\psi} = \psi \mathbf{k}$ and ψ is commonly known as the stream function. It follows that

$$u = \frac{\partial \psi}{\partial y} \quad \text{and} \quad v = -\frac{\partial \psi}{\partial x} \quad (11)$$

With this setting the mass conservation equation becomes trivially satisfied, as shown here

$$\nabla \cdot \mathbf{u} = \frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = \frac{\partial^2 \psi}{\partial y \partial x} - \frac{\partial^2 \psi}{\partial y \partial x} = 0 \quad (12)$$

The relation between the stream function and the vorticity in two dimensions can be found by substituting equations (11) into equation (8). This yields

$$\nabla^2 \psi = -\omega \quad (13)$$

1.3 Flow model

Combining equations (9, 13) results in the *Psi-Zita* model, namely

$$\begin{cases} \frac{\partial \zeta}{\partial t} + \mathbf{u} \cdot \nabla \zeta = \nu \nabla^2 \zeta \\ \nabla^2 \psi = \zeta \end{cases} \quad (14)$$

where the new flow variable $\zeta = -\omega$ has been defined and will be used from now on.

An adimensionalization can be done on the set of equations (14) by introducing constant reference dimensional values, denoted with a subscript «ref», and dimensionless variables denoted with a tilde. In particular, these new positions are considered

$$\mathbf{u} = u_{\text{ref}} \tilde{\mathbf{u}} \quad (15)$$

$$\zeta = \frac{u_{\text{ref}}}{L_{\text{ref}}} \tilde{\zeta} \quad (16)$$

$$\psi = L_{\text{ref}} u_{\text{ref}} \tilde{\psi} \quad (17)$$

$$t = \frac{L_{\text{ref}}}{u_{\text{ref}}} \tilde{t} \quad (18)$$

$$\nabla = \frac{1}{L_{\text{ref}}} \tilde{\nabla} \quad (19)$$

$$\nabla^2 = \frac{1}{L_{\text{ref}}^2} \tilde{\nabla}^2 \quad (20)$$

where (i) u_{ref} is the reference speed, (ii) L_{ref} is the reference length, (iii) $\tilde{\mathbf{u}}$ is the adimensional velocity vector field, (iv) $\tilde{\zeta}$ is the opposite of the adimensional vorticity, (v) $\tilde{\psi}$ is the adimensional stream function, and (vi) $\tilde{t}$ is the adimensional time. In addition, also the dimensionless gradient and Laplacian operator have been defined, those are $\tilde{\nabla}$ and $\tilde{\nabla}^2$ respectively.

This adimensional setup allows to rewrite the ψ - ζ model as follows

$$\begin{cases} \frac{\partial \tilde{\zeta}}{\partial \tilde{t}} + \tilde{\mathbf{u}} \cdot \tilde{\nabla} \tilde{\zeta} = \frac{1}{\text{Re}} \tilde{\nabla}^2 \tilde{\zeta} \\ \tilde{\nabla}^2 \tilde{\psi} = \tilde{\zeta} \end{cases} \quad (21)$$

with Re being the Reynolds' number. It indicates the ratio between inertial and viscous forces and in this context it has been defined as

$$\text{Re} = \frac{u_{\text{ref}} L_{\text{ref}}}{\nu} \quad (22)$$

2 Lines of flow

In addition to knowing the physical parameters of a fluid flow, it can be useful to draw pictures of specific curves that give an intuitive understanding of the motion. Those curves are so called lines of flow and they can be of three different types, with their own definitions and usage cases.

A discussion of their mathematical characterization and some examples and applications follow below.

2.1 Streamlines, pathlines and streaklines

A *streamline* is a curve whose tangent at any point is in the direction of the velocity vector at that point. In particular, a *streamline* can be mathematically defined as follows [1]

$$\mathbf{u} \times d\boldsymbol{\ell} = 0 \quad (23)$$

where $d\boldsymbol{\ell}$ is the directed element of the *streamline*. This relation corresponds to the

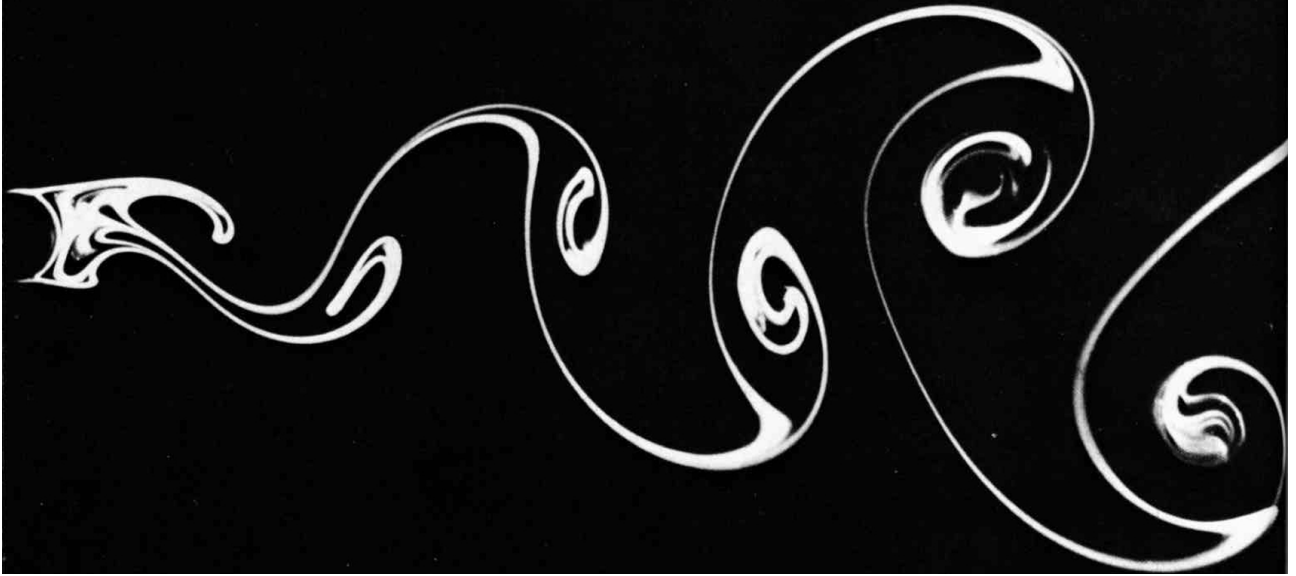


Figure 1 Kármán vortex street behind a circular cylinder [2]. *Streaklines* are the easiest lines of flow to visualize during experimental procedures. They can be obtained by adding dye at a specific position, then the fluid flows and transports the dye realizing colored and well distinguishable *streaklines*.

tangent and the velocity being parallel at every point.

A *pathline* is the trajectory that a fluid particle follows through its motion. It is defined as the curve $x_q = x_q(t)$ satisfying the mathematical relation

$$\frac{dx_q}{dt} = u(x_q(t), t) \quad (24)$$

with q being an arbitrary fluid particle and x_q its position.

It can easily be noticed that the trajectory of q is determined by the Eulerian velocity vector field $u(x, t)$, thereby every *pathline* depends on the *streamlines*.

Pathlines and *streamlines* are different curves for unsteady flows. In the general case *streamlines* represent a snapshot of the flow at any given instant t , since the velocity vector field $u(x, t)$ explicitly depends on time, whereas *pathlines* give an overview on the previous history of the flow.

A *streakline* is a curve $x_s = x_s(t)$ joining the isochronous positions of all the particles that have passed through a reference point x_0 at a previous instant. Those isochronous positions $x_{q_k}(t)$ can be found by

solving the following differential equations, together with its initial condition

$$\begin{cases} \frac{dx_{q_k}}{dt} = u(x_{q_k}(t), t) \\ x_{q_k}(t_0 + k\Delta t) = x_0 \end{cases} \quad (25)$$

where q_k is the k -th tracked particle that passed through x_0 at time $t_0 + k\Delta t$.

It is worth noticing that to evaluate the *streaklines*, one can before compute all the required *pathlines* $x_{q_k} = x_{q_k}(t)$ that have a common initial position at different initial instants.

Figure 1 shows a real visualization of *streaklines* in the wake of a cylinder, that has been photographed during an experiment.

3 Flow simulation

In this section of the study all the actual simulation concepts and data will be presented and discussed.

The simulation will integrate the set of equations ψ - ζ (21) to solve fluid flow inside a rectangular two-dimensional duct containing a circular cylinder as obstacle. Therefore,

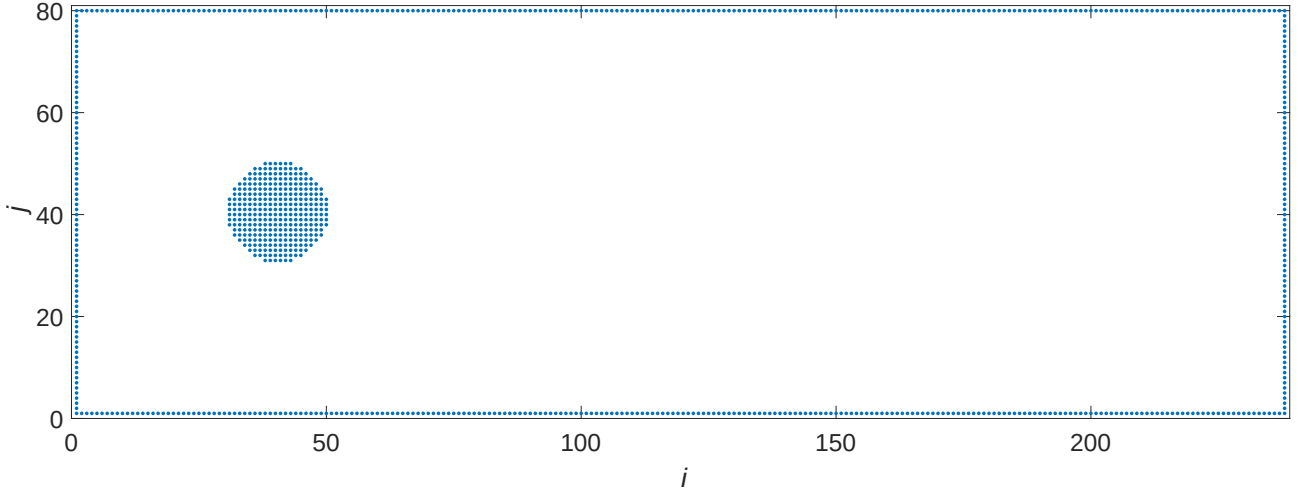


Figure 2 Discretized mesh for the flow around cylinder simulation. Blue nodes represent non-flowing zones, in this case the boundaries and the inside of the circular cylinder, and white space represents where the fluid is allowed to flow. As the diagram shows, indices i and j distinguish every node of the mesh, ordering them in equivalence with Cartesian coordinates x and y respectively.

anything will be carried out with dimensionless physical quantities, as the model requires.

For ease of notation tildes will be omitted from the adimensional variables hence quantities x, u, ζ, ψ , etc. must be considered as pure numbers from now on.

Here some new parameters characterizing the simulation are introduced, namely (i) L_x and L_y are the rectangular domain dimensions, (ii) h is the cell spacing of the mesh, the same for both the x and y directions, (iii) N_x and N_y are the number of nodes in the x and y directions respectively, (iv) T is the ending time of the simulation, (v) Δt is the timestep width, and (vi) N_t is the number of timesteps.

In addition, to account for stability two parameters are used. The convective stability parameter and the diffusive stability parameter, respectively the Courant's number Cou and the β number. In this context they are defined as

$$Cou = \frac{u_{ref}\Delta t}{h} \quad \beta = \frac{\Delta t}{h^2 Re} \quad (26)$$

Cou and β will be assigned at sufficiently low values and the temporal discretization

will be found by evaluating the timestep width as

$$\Delta t = \min \left\{ Cou \frac{h}{u_{ref}}, \beta h^2 Re \right\} \quad (27)$$

3.1 Domain

The domain is discretized with an uniform mesh of cell spacing $\Delta x = \Delta y = h$. To do so, the physical domain is defined using an aspect ratio parameter. In particular, after assigned the short side of the domain L_y , the other one can be obtained using

$$L_x = \mathcal{R}L_y \quad (28)$$

with $\mathcal{R} > 1$. This allows to define the mesh in the same way, by first assigning the number of nodes in the y direction and then compute the number of nodes in the x direction. In particular, the following relation must be used

$$(N_x - 1) = \mathcal{R}(N_y - 1) \quad (29)$$

where $(N_x - 1)$ and $(N_y - 1)$ are the number of cells in the x and y directions respectively. This effectively allows the mesh to be equally spaced.

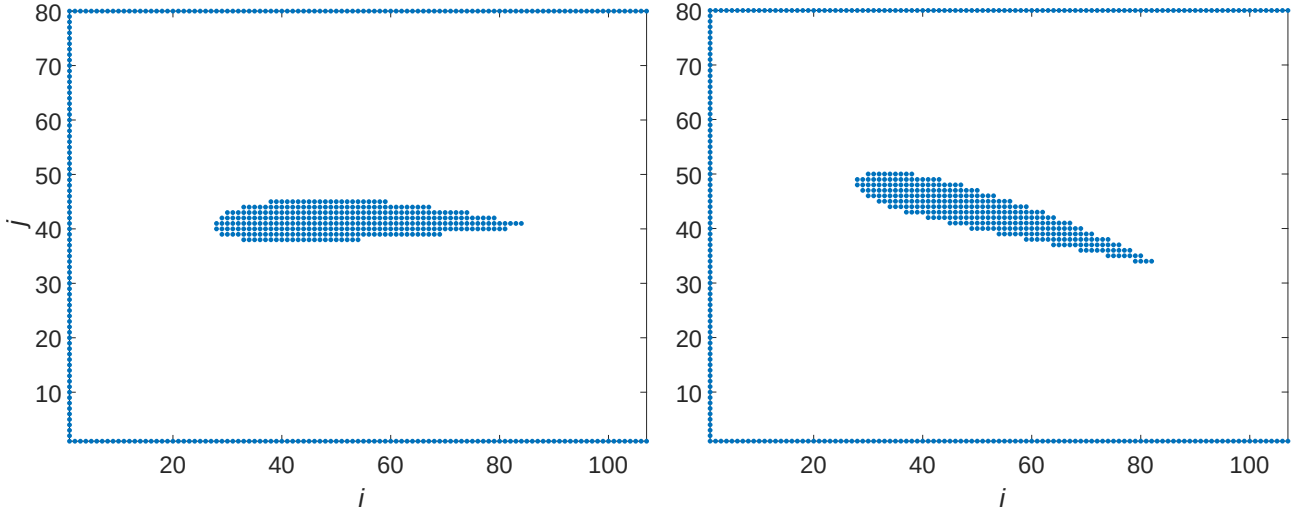


Figure 3 Discretized meshes for the flow around airfoil simulation. The shape of a 2418 NACA airfoil has been approximated at angles 0 deg and 15 deg. Same mesh conventions already introduced with figure 2 are assumed here. Increasing N_y will eventually result in more accurate shapes.

Algorithm 1 Assignment of a topological matrix G to enumerate mesh nodes in case of a cylindrical obstacle.

Data: Mesh sizes N_x and N_y
Position of every node x_{ij}
Cylinder center x_C
Cylinder radius R

```

k = 0;
G = ZeroMatrix( $N_x \times N_y$ );
for j = 2, 3, 4, ..., ( $N_y - 1$ ) do
    for i = 2, 3, 4, ..., ( $N_x - 1$ ) do
        if  $|x_{ij} - x_C|^2 \leq R^2$  then
             $G_{ij} = -1$ ;
        else
            k = k + 1;
             $G_{ij} = k$ ;
        end
    end
end
end

```

As shown in figures 2 and 3, nodes in the mesh are ordered in equivalence with Cartesian coordinates x and y . In particular, indices i and j are used to indicate a generic node n sitting at position $x_n = (x_i, y_j)$, with the discrete coordinate values x_i and y_j being monotonically increasing.

To take into account the non-flowing zones of the domain, a topological matrix G can be defined, having N_x rows and N_y columns, such that each of its elements cor-

responds to a node of the mesh. The elements corresponding to boundary nodes are assigned as zero, the elements corresponding to the inside of the obstacle are assigned as -1 , and the elements corresponding to flowing zone nodes are assigned as a counting index k , starting from 1 and ending at the last available node. Algorithms 1 and 2 show how such matrix can be computed.

3.2 Boundary conditions

To accurately solve for the fluid flow, it is crucial to assign appropriate boundary conditions in the variables u , ψ and ζ .

At west and east walls uniform axial inflow and outflow are assigned, thereby for the velocity vector field a Dirichlet condition is used, namely

$$u_W = u_E = u_{\text{ref}} = [u_{\text{ref}}, 0]^T \quad (30)$$

where u_{ref} is evaluated from an assigned fluid volume flow $Q = u_{\text{ref}}/L_y$.

To take into account the required fluid flow, the stream function must be assigned here as a linear function of y , from a value of 0 to a value of Q . In particular,

$$\psi_W = \psi_E = \frac{Q}{L_y} y \quad (31)$$

Algorithm 2 Assignment of a topological matrix G to enumerate mesh nodes in case of an airfoil obstacle [3].

Data: Mesh sizes N_x and N_y
 Position of every node $\mathbf{x}_{ij} = (x_{ij}, y_{ij})$
 Leading edge position $\mathbf{x}_{le} = (x_{le}, y_{le})$
 Trailing edge position $\mathbf{x}_{te} = (x_{te}, y_{te})$
 Profile equations $y_U(x_c)$ and $y_L(x_c)$

```

k = 0;
G = ZeroMatrix( $N_x \times N_y$ );
for j = 2, 3, 4, ..., ( $N_y - 1$ ) do
  for i = 2, 3, 4, ..., ( $N_x - 1$ ) do
    if  $x_{le} \leq x_{ij} \leq x_{le} + c$  then
       $x_{ij,c} = \frac{(\mathbf{x}_{ij} - \mathbf{x}_{le}) \cdot (\mathbf{x}_{te} - \mathbf{x}_{le})}{|\mathbf{x}_{te} - \mathbf{x}_{le}|^2}$ ;
      if  $y_L(x_{ij,c}) \leq y_{ij} \leq y_U(x_{ij,c})$  then
         $G_{ij} = -1$ ;
      end
    else
      k = k + 1;
       $G_{ij} = k$ ;
    end
  end
end
end

```

South and North walls are threatened as *streamlines*, thereby for the stream function a Dirichlet boundary condition is used, namely

$$\psi_S = 0 \quad \text{and} \quad \psi_N = Q \quad (32)$$

Interfaces with the obstacle are threatened as streamlines, with an assigned Dirichlet boundary condition

$$\psi_O = \frac{Q}{2} \quad (33)$$

such that the volume flow rate on the sides is exactly $2 \cdot Q/2 = Q$, thereby the conservation of mass is satisfied.

In addition, south, north, and obstacle walls are threatened with a no-slip condition for the velocity boundary condition. Therefore, given a wall w with directed element $d\ell$, the following relation is imposed

$$\mathbf{u}_w \cdot d\ell = 0 \quad (34)$$

To decide what values to assign to ζ at the boundaries, a Taylor series expansion of

the stream function can be used, namely

$$\psi_n = \psi_w + \left. \frac{\partial \psi}{\partial y} \right|_w h + \left. \frac{\partial^2 \psi}{\partial y^2} \right|_w \frac{h^2}{2} + \mathcal{O}(h^3) \quad (35)$$

where n is a mesh node near to a wall node w . Substituting equation (11) and the second equation of (21) yields

$$\psi_n = \psi_w + u_w h + \zeta_w \frac{h^2}{2} + \mathcal{O}(h^3) \quad (36)$$

if the examined wall is threatened as a streamline as in this case. This allows to obtain the Dirichlet boundary condition for ζ by rearranging

$$\zeta_w = \frac{2(\psi_n - \psi_w - u_w h)}{h^2} + \mathcal{O}(h) \quad (37)$$

It is worth noticing that formula (37) is suitable for a south wall, but can be easily generalized for an arbitrary oriented wall. Formulae obtained with that same approach are known as Thom's formulae.

In case of a no-slip boundary condition on the velocity, equation (37) can be further simplified using $u_w = 0$, yielding

$$\zeta_w = \frac{2(\psi_n - \psi_w)}{h^2} + \mathcal{O}(h) \quad (38)$$

that is the one required in this context.

3.3 Discretization of diffusive and convective terms

To solve the differential problem (21), unknowns ζ and ψ are substituted with their discrete counterparts ζ and ψ . Those are column vector containing the values ζ and ψ assume at each node of the mesh, ordered as specified by algorithm 1.

Similarly, every differential operator D acting on ζ or ψ will be substituted with its discrete form. It will act on vectors ζ or ψ , thereby it will be a matrix D .

Diffusive term $\nabla^2 \zeta$ in the first equation of (21) can be discretized with a discrete Laplacian operator L such that $\nabla^2 \zeta$ will be substituted with $L\zeta$. It can be obtained by

using finite difference central schemes for the second order derivatives. In particular,

$$\left. \frac{\partial^2 \zeta}{\partial x^2} \right|_{ij} = \frac{\zeta_{i+1,j} - 2\zeta_{ij} + \zeta_{i-1,j}}{h^2} + \mathcal{O}(h^2) \quad (39)$$

$$\left. \frac{\partial^2 \zeta}{\partial y^2} \right|_{ij} = \frac{\zeta_{i,j+1} - 2\zeta_{ij} + \zeta_{i,j-1}}{h^2} + \mathcal{O}(h^2) \quad (40)$$

yielding

$$\nabla^2 \zeta|_{ij} = \frac{1}{h^2} (\zeta_{i+1,j} + \zeta_{i-1,j} + \zeta_{i,j+1} + \zeta_{i,j-1} - 4\zeta_{ij}) + \mathcal{O}(h^2) \quad (41)$$

Algorithm 3 shows how such matrix L can be computed.

Algorithm 3 Example of definition of the discrete Laplacian operator L .

Data: Mesh sizes N_x and N_y
 Nodes spacing h
 Topological matrix G
 Number of internal nodes N

```

 $L = \text{ZeroMatrix}(N \times N);$ 
for  $j = 2, 3, 4, \dots, (N_y - 1)$  do
  for  $i = 2, 3, 4, \dots, (N_x - 1)$  do
    if  $G_{ij} > 0$  then
       $k = G_{ij};$ 
       $L_{kk} = -4/h^2;$ 
       $k_E = G_{i+1,j};$ 
       $k_W = G_{i-1,j};$ 
       $k_N = G_{i,j+1};$ 
       $k_S = G_{i,j-1};$ 
      if  $k_E > 0$  then
         $L_{kk_E} = 1/h^2;$ 
      end
      if  $k_W > 0$  then
         $L_{kk_W} = 1/h^2;$ 
      end
      if  $k_N > 0$  then
         $L_{kk_N} = 1/h^2;$ 
      end
      if  $k_S > 0$  then
         $L_{kk_S} = 1/h^2;$ 
      end
    end
  end
end

```

The convective term $\mathbf{u} \cdot \nabla \zeta = \nabla \cdot (\mathbf{u} \zeta)$ in the first equation of (21) can be discretized

following various approaches. It can be seen in the advective form $\mathbf{u} \cdot \nabla \zeta$, in the divergence form $\nabla \cdot (\mathbf{u} \zeta)$, or in any appropriate linear combination of them.

The advective form of the convective term gives the following discretization

$$(\mathbf{U} D_x + \mathbf{V} D_y) \zeta \quad (42)$$

whereas the divergence form yields

$$(\mathbf{D}_x \mathbf{U} + \mathbf{D}_y \mathbf{V}) \zeta \quad (43)$$

where (i) $\mathbf{U} = \text{diag}(u_k)$, (ii) $\mathbf{V} = \text{diag}(v_k)$, and (iii) $\mathbf{D}_x$ and $\mathbf{D}_y$ are the discrete first order derivative operators in the x and y directions respectively.

An appropriate linear combination of the above two forms of the convective term is

$$\alpha \mathbf{u} \cdot \nabla \zeta + \beta \nabla \cdot (\mathbf{u} \zeta) \quad (44)$$

with $\alpha + \beta = 1$. This yields the following discretization

$$[\alpha (\mathbf{U} D_x + \mathbf{V} D_y) + \beta (\mathbf{D}_x \mathbf{U} + \mathbf{D}_y \mathbf{V})] \zeta \quad (45)$$

To decide which discretization approach to follow, it is useful to introduce the total enstrophy $\mathcal{E}$ over the domain Ω . In this context it is defined as

$$\mathcal{E} = \int_{\Omega} \frac{\zeta^2}{2} d\Omega \quad (46)$$

and it must conserve, thereby the condition $d\mathcal{E}/dt = 0$ must be satisfied. In particular,

$$\int_{\Omega} \left[\frac{\partial}{\partial t} \left(\frac{\zeta^2}{2} \right) + \mathbf{u} \cdot \nabla \left(\frac{\zeta^2}{2} \right) \right] d\Omega = 0 \quad (47)$$

having used Reynolds' transport equation.

Since the domain Ω is stationary, both of the integrals that arise in equation (47) must be zero. This gives the following relation

$$\int_{\Omega} \mathbf{u} \cdot \nabla \left(\frac{\zeta^2}{2} \right) d\Omega = \int_{\Omega} \zeta \mathbf{u} \cdot \nabla \zeta d\Omega = 0 \quad (48)$$

that when discretized is equivalent to

$$\zeta^T [\alpha (\mathbf{U} D_x + \mathbf{V} D_y) + \beta (\mathbf{D}_x \mathbf{U} + \mathbf{D}_y \mathbf{V})] \zeta = 0 \quad (49)$$

assuming the most general form of the convective term.

The imposed condition (49) is equivalent to require a quadratic form to always be zero, thereby it is equivalent to require the matrix of the form to be anti-symmetrical. This can be achieved only when

$$\alpha = \beta = 1/2 \quad (50)$$

that gives

$$\frac{1}{2} (\mathbf{U} \mathbf{D}_x + \mathbf{V} \mathbf{D}_y + \mathbf{D}_x \mathbf{U} + \mathbf{D}_y \mathbf{V}) \zeta \quad (51)$$

as the most appropriate discretization of the convective term to satisfy the conservation of enstrophy.

Discretization (51) is known as the skew-symmetric form of the discretized convective term and from now on its matrix operator will be denoted as

$$\mathbf{C} = \frac{1}{2} (\mathbf{U} \mathbf{D}_x + \mathbf{V} \mathbf{D}_y + \mathbf{D}_x \mathbf{U} + \mathbf{D}_y \mathbf{V}) \quad (52)$$

3.4 Time integration

Using the spatial discretizations (41, 51) of the diffusive and convective terms, the set of partial differential equations (21) becomes the following set of constrained ordinary differential equations

$$\begin{cases} \frac{d\zeta}{dt} = -\mathbf{C}\zeta + \frac{1}{\text{Re}} \mathbf{L}\zeta \\ \mathbf{L}\psi = \zeta \end{cases} \quad (53)$$

that can be integrated using a numerical procedure, e.g. using a multi-step method or a single-step multi-stage method.

In this context a 4th order explicit Runge-Kutta method [6] is used to integrate the differential problem, given by the following Butcher tableau

$$\begin{array}{c|ccc} 0 & 0 & & & \\ 1/2 & 1/2 & 0 & & \\ 1/2 & 0 & 1/2 & 0 & \\ 1 & 0 & 0 & 1 & 0 \\ \hline & 1/6 & 1/3 & 1/3 & 1/6 \end{array} \quad (54)$$

Algorithm 4 A Runge-Kutta iteration to solve for ζ and ψ through integration over a timestep.

Data: Timestep width Δt
Discrete Laplacian operator $\mathbf{L}$
Convective discretization operator $\mathbf{C}$
Previous timestep value ζ_{old}
Boundary conditions for ζ

$\zeta_1 = \zeta_{\text{old}};$
 $f_1 = -\mathbf{C}\zeta_1 + \frac{1}{\text{Re}} \mathbf{L}\zeta_1;$
 $\zeta_2 = \zeta_{\text{old}} + \frac{\Delta t}{2} f_1;$
 $f_2 = -\mathbf{C}\zeta_2 + \frac{1}{\text{Re}} \mathbf{L}\zeta_2;$
 $\zeta_3 = \zeta_{\text{old}} + \frac{\Delta t}{2} f_2;$
 $f_3 = -\mathbf{C}\zeta_3 + \frac{1}{\text{Re}} \mathbf{L}\zeta_3;$
 $\zeta_4 = \zeta_{\text{old}} + \Delta t f_3;$
 $f_4 = -\mathbf{C}\zeta_4 + \frac{1}{\text{Re}} \mathbf{L}\zeta_4;$
 $\zeta_{\text{new}} = \zeta_{\text{old}} + \Delta t \left[\frac{1}{6} (f_1 + f_4) + \frac{1}{3} (f_2 + f_3) \right];$
Adjust ζ_{new} for boundary conditions;
 $\psi_{\text{new}} = \mathbf{L}^{-1} \zeta_{\text{new}};$

At each timestep the first set of equations in (53) is integrated through four stages using the Runge-Kutta method provided by the Butcher tableau (54). Then, the second set of linear equations can be solved as follows

$$\psi = \mathbf{L}^{-1} \zeta \quad (55)$$

and the procedure can be repeated after adjusting for the required boundary conditions.

The algorithm that can carry out such computation for a single iteration is shown here

In order to compute the *streaklines*, after solving for ζ and ψ also sets of equations (25) must be integrated at each timestep, with k ranging from 0 to the timestep index i_t . To do so the velocity vector field must be evaluated from the stream function discretizing equations (11).

Algorithm 5 A Runge-Kutta iteration to solve for x_{q_k} through integration over a timestep.

Data: Timestep width Δt
 Previous timestep value x_{old}^k
 u instantaneous node values $\mathbf{U}$
 v instantaneous node values $\mathbf{V}$
 An interpolation procedure

```

 $x_1^k = x_{\text{old}}^k;$ 
 $f_1 = \text{Interpolate}(\mathbf{U}, \mathbf{V}, x_1^k);$ 
 $x_2^k = x_{\text{old}}^k + \frac{\Delta t}{2} f_1;$ 
 $f_2 = \text{Interpolate}(\mathbf{U}, \mathbf{V}, x_2^k);$ 
 $x_3^k = x_{\text{old}}^k + \frac{\Delta t}{2} f_2;$ 
 $f_3 = \text{Interpolate}(\mathbf{U}, \mathbf{V}, x_3^k);$ 
 $x_4^k = x_{\text{old}}^k + \Delta t f_3;$ 
 $f_4 = \text{Interpolate}(\mathbf{U}, \mathbf{V}, x_4^k);$ 
 $x_{\text{new}}^k = x_{\text{old}}^k + \Delta t \left[ \frac{1}{6} (f_1 + f_4) + \frac{1}{3} (f_2 + f_3) \right];$ 

```

This can be achieved using finite difference central schemes for the first order derivative, namely

$$u_{ij} = \frac{\psi_{i,j+1} - \psi_{i,j-1}}{2h} + \mathcal{O}(h^2) \quad (56)$$

$$v_{ij} = -\frac{\psi_{i+1,j} - \psi_{i-1,j}}{2h} + \mathcal{O}(h^2) \quad (57)$$

Since the right-hand side of the k -th set of equations (25) for the k -th *pathline* is $\mathbf{u}(x_{q_k}, t)$, the velocity vector field must be interpolated from its node values to the required particle position x_{q_k} . This can be achieved by a trivial bi-linear interpolation, as shown below

$$u_{q_k} = \begin{bmatrix} 1 - \sigma_x & \sigma_x \end{bmatrix} \begin{bmatrix} u_{\text{SW}} & u_{\text{NW}} \\ u_{\text{SE}} & u_{\text{NE}} \end{bmatrix} \begin{bmatrix} 1 - \sigma_y \\ \sigma_y \end{bmatrix} \quad (58)$$

$$v_{q_k} = \begin{bmatrix} 1 - \sigma_x & \sigma_x \end{bmatrix} \begin{bmatrix} v_{\text{SW}} & v_{\text{NW}} \\ v_{\text{SE}} & v_{\text{NE}} \end{bmatrix} \begin{bmatrix} 1 - \sigma_y \\ \sigma_y \end{bmatrix} \quad (59)$$

where

$$\sigma_x = \frac{x_{q_k} - x_i}{h} \quad \sigma_y = \frac{y_{q_k} - y_j}{h} \quad (60)$$

and subscripts «SW», «NW», «SE», and «NE» refer to the values that u and v assume at the nodes of the cell containing particle q_k .

At each timestep the particle position x_{q_k} can be integrated with the same Runge-Kutta method (54) already used. An example of an algorithm that interpolates and integrates over a single timestep to solve for the k -th particle position is algorithm 5.

From the results obtained at each timestep and for every *pathline* x^k with algorithm 5, it is possible to draw the relative *streakline* by joining all most recent positions.

3.5 Implementation

Three simulations will be carried out: one with the obstacle being a circular cylindrical, one with it being a 2418 NACA airfoil at angle $\alpha = 0$ deg, and one near the stall, at angle $\alpha = 15$ deg.

To achieve those MATLAB has being used, with two main scripts, suitable for each case, and some useful routines to ease the management of the code.

As shown below, a list of the most important MATLAB files is included here. Some graphics routine has been omitted, since they are not much of interest. Also the main script used for the second and third simulations are not shown, since it can be obtained from the first by only changing the computation of the topological matrix G . In particular, an approach based on algorithm 2 should be followed, code 8 achieves this.

In addition, during the first simulation the history of the total enstrophy previously defined in (46) is computed at each timestep, to later plot it and see if its conservation is satisfied. It is numerically computed with the following integral approximation

$$\mathcal{E}(t_i) \approx h^2 \sum_{i=1}^{N_x} \sum_{j=1}^{N_y} \zeta_{ij}(t_i), \quad i_t = 1, 2, \dots, N_t \quad (61)$$

The geometrical data chosen for these simulations domain are: (i) $L_y = 1$, (ii) $N_y = 80$, and (iii) $\mathcal{R} = 3$. Time and stability chosen parameters are mentioned and shown in code 1.

Code 1 FlowAroundCylinder.m. Main script that simulates the flow around a cylinder of assigned size and position. $Re = 500$, $Cou = 0.3$, $\beta = 0.5$, and $T = 30$.

```

1 close all; clear; clc
2
3 % Domain geometry
4 Ly = 1;
5 AR = 3;
6 Lx = AR * Ly;
7 Nhy = 79;
8 Nhx = AR * Nhy;
9 Ny = Nhy + 1;
10 Nx = Nhx + 1;
11 y = linspace(0, Ly, Ny);
12 x = linspace(0, Lx, Nx);
13 h = x(2) - x(1);
14 hq = h * h;
15 [X, Y] = meshgrid(x, y);
16
17 % Boundary conditions
18 Q = 1;
19 PsiN = Q * ones(1, Nx)';
20 PsiS = zeros(1, Nx)';
21 PsiW = linspace(0, Q, Ny);
22 PsiE = PsiW;
23 PsiCyl = Q / 2;
24
25 % Physical parameters
26 T = 30;
27 Re = 500;
28 Cou = 0.3;
29 beta = 0.5;
30 uRef = Q / Ly;
31 Dt = min(Cou * h / uRef, beta * Re * hq);
32 Nt = ceil(T / Dt) + 1;
33 t = linspace(0, T, Nt);
34 Dt = t(2) - t(1);
35
36 % Obstacle geometry
37 Center = [Lx/6, Ly/2];
38 Radius = Ly / 8;
39
40 % Domain topology and discrete Laplacian operator
41 k = 0;
42 G = zeros(Nx, Ny);
43 for j = 2 : Ny-1
44     for i = 2 : Nx-1
45         if ([x(i), y(j)] - Center) * ([x(i), y(j)] - Center)' <= Radius^2
46             G(i, j) = 0;
47         else
48             k = k + 1;
49             G(i, j) = k;

```

```
50         end
51     end
52 end
53 Lap = -delSq(G) ./ hq;
54 for i = 2 : Nx-1
55     for j = 2 : Ny-1
56         if G(i,j) == 0
57             G(i,j) = -1;
58         end
59     end
60 end
61
62 % Domain visualization
63 figurePosition = [0, 50, 1640, 1640/AR];
64 f = figure(Position=figurePosition, Units="pixels");
65 D = zeros(Nx, Ny);
66 D(G == -1) = 1;
67 D(G == 0) = 1;
68 D = D';
69 spy(D)
70 xlabel('\it i')
71 ylabel('\it j')
72 set(gca, "YDir", "normal", "FontSize", 18)
73 exportgraphics(f, 'Domain.pdf', Resolution=300)
74
75 % Array initialization
76 zita = zeros(k, 1);
77 PSI = zeros(Nx, Ny);
78 ZITA = PSI;
79 U = PSI;
80 V = PSI;
81
82 % Boundary conditions into the arrays
83 U(1, :) = uRef;
84 U(end, :) = uRef;
85 PSI(1, :) = PsiW;
86 PSI(end, :) = PsiE;
87 PSI(:, 1) = PsiS;
88 PSI(:, end) = PsiN;
89 PSI(G == -1) = PsiCyl;
90
91 % Pathlines initial condition (point of injection)
92 xP{1} = [0, Ly/2+0.100];
93 xP{2} = [0, Ly/2+0.125];
94 xP{3} = [0, Ly/2+0.150];
95 xP{4} = [0, Ly/2-0.100];
96 xP{5} = [0, Ly/2-0.125];
97 xP{6} = [0, Ly/2-0.150];
98
99 % Streaklines array initialization
100 xS{1} = zeros(2, Nt);
101 xS{2} = zeros(2, Nt);
102 xS{3} = zeros(2, Nt);
103 xS{4} = zeros(2, Nt);
104 xS{5} = zeros(2, Nt);
105 xS{6} = zeros(2, Nt);
```

```

106 for it = 1 : Nt
107     xS{1}(:, it) = xP{1};
108     xS{2}(:, it) = xP{2};
109     xS{3}(:, it) = xP{3};
110     xS{4}(:, it) = xP{4};
111     xS{5}(:, it) = xP{5};
112     xS{6}(:, it) = xP{6};
113 end
114
115 % Graphics preparation
116 f = figure(Position=figurePosition, Units="pixels");
117 Uquiv = zeros(Nx, Ny);
118 Vquiv = zeros(Nx, Ny);
119 iq = 1 : 3 : Nx;
120 jq = [1 : 6 : floor(Ny/3), ...
121       floor(Ny/3+3) : 3 : floor(2*Ny/3), floor(2*Ny/3+6) : 6 : Ny];
122 Map = [linspace(75/255, 1, 500)', linspace(90/255, 1, 500)', ...
123       linspace(241/255, 1, 500)', linspace(1, 180/255, 500)', ...
124       linspace(1, 10/255, 500)', linspace(1, 32/255, 500)'];
125 DeltaPictures = round(Nt / 20, 1, "significant");
126
127 % Evolution and graphics
128 for it = 1 : Nt
129     time = t(it);
130     ZITA(G == -1) = 0;
131     ZITA = ThomFormulae(ZITA, PSI, G, hq);
132     ZITA = RK4(time, ZITA, ...
133               @(t_, y_) ZitaRHS(t_, y_, Re, h, hq, U, V, G, Nx, Ny), ...
134               Dt);
135     [PSI, zita] = zita2psi(zita, ZITA, PSI, Lap, G, Nx, Ny, hq, ...
136                           PsiS, PsiN, PsiW, PsiE, PsiCyl);
137     [U, V] = psi2uv(PSI, U, V, G, h, Nx, Ny, uRef);
138     for k = 1 : it
139         xS{1}(:, k) = ...
140             RK4(time, xS{1}(:, k), @(t_, y_) ...
141                 PathlinesRHS(t_, y_, x, y, U, V, h, h, Nx, Ny), ...
142                 Dt);
143         xS{2}(:, k) = ...
144             RK4(time, xS{2}(:, k), @(t_, y_) ...
145                 PathlinesRHS(t_, y_, x, y, U, V, h, h, Nx, Ny), ...
146                 Dt);
147         xS{3}(:, k) = ...
148             RK4(time, xS{3}(:, k), @(t_, y_) ...
149                 PathlinesRHS(t_, y_, x, y, U, V, h, h, Nx, Ny), ...
150                 Dt);
151         xS{4}(:, k) = ...
152             RK4(time, xS{4}(:, k), @(t_, y_) ...
153                 PathlinesRHS(t_, y_, x, y, U, V, h, h, Nx, Ny), ...
154                 Dt);
155         xS{5}(:, k) = ...
156             RK4(time, xS{5}(:, k), @(t_, y_) ...
157                 PathlinesRHS(t_, y_, x, y, U, V, h, h, Nx, Ny), ...
158                 Dt);
159         xS{6}(:, k) = ...
160             RK4(time, xS{6}(:, k), @(t_, y_) ...
161                 PathlinesRHS(t_, y_, x, y, U, V, h, h, Nx, Ny), ...

```

```

162         Dt);
163     end
164     indices = linspace(1, it, 20000);
165     if mod(it, 10) == 0
166         ZITA(G == -1) = NaN;
167         pcolor(x, y, ZITA');
168         colormap(Map)
169         shading interp
170         colorbar
171         clim([0.7*min(min(ZITA(G > 0))), 0.7*max(max(ZITA(G > 0)))]])
172         hold on
173         Uquiv(iq, jq) = U(iq, jq);
174         Vquiv(iq, jq) = V(iq, jq);
175         quiver(X, Y, Uquiv', Vquiv', 3, Color='k')
176         rectangle( ...
177             Position=[Center-Radius*2.145/2, 2.145*Radius, 2.145*Radius], ...
178             Curvature=[1, 1], ...
179             LineWidth=2.5, FaceColor='k')
180         xSplot{1} = interp1(1:it, xS{1}(1, 1:it), indices, "pchip");
181         ySplot{1} = interp1(1:it, xS{1}(2, 1:it), indices, "pchip");
182         xSplot{2} = interp1(1:it, xS{2}(1, 1:it), indices, "pchip");
183         ySplot{2} = interp1(1:it, xS{2}(2, 1:it), indices, "pchip");
184         xSplot{3} = interp1(1:it, xS{3}(1, 1:it), indices, "pchip");
185         ySplot{3} = interp1(1:it, xS{3}(2, 1:it), indices, "pchip");
186         xSplot{4} = interp1(1:it, xS{4}(1, 1:it), indices, "pchip");
187         ySplot{4} = interp1(1:it, xS{4}(2, 1:it), indices, "pchip");
188         xSplot{5} = interp1(1:it, xS{5}(1, 1:it), indices, "pchip");
189         ySplot{5} = interp1(1:it, xS{5}(2, 1:it), indices, "pchip");
190         xSplot{6} = interp1(1:it, xS{6}(1, 1:it), indices, "pchip");
191         ySplot{6} = interp1(1:it, xS{6}(2, 1:it), indices, "pchip");
192         plot(xSplot{1}, smooth(ySplot{1}), LineWidth=1.5, Color='r')
193         plot(xSplot{2}, smooth(ySplot{2}), LineWidth=1.5, Color='r')
194         plot(xSplot{3}, smooth(ySplot{3}), LineWidth=1.5, Color='r')
195         plot(xSplot{4}, smooth(ySplot{4}), LineWidth=1.5, Color='b')
196         plot(xSplot{5}, smooth(ySplot{5}), LineWidth=1.5, Color='b')
197         plot(xSplot{6}, smooth(ySplot{6}), LineWidth=1.5, Color='b')
198         title([ ...
199             '\it t} = ', num2str(time, 2), '. {\it i_t} = ', ...
200             num2str(it), ' of ', num2str(Nt) ...
201             ], FontWeight="normal")
202         axis image
203         axis([0, Lx, 0, Ly])
204         xlabel('\it x')
205         ylabel('\it y')
206         set(gca, "FontSize", 18)
207         drawnow limitrate
208         hold off
209     end
210 end

```

Code 2 ThomFormulae.m. Routine that computes boundary conditions for ζ using Thom's formulae.

```

1 function ZITA = ThomFormulae(ZITA, PSI, G, hq)
2
3 [Nx, Ny] = size(ZITA);
4
5 i = 2 : Nx - 1;
6 j = 1;
7 ZITA(i, j) = 2 * (PSI(i, j+1) - PSI(i, j)) / hq;
8
9 i = 2 : Nx - 1;
10 j = Ny;
11 ZITA(i, j) = 2 * (PSI(i, j-1) - PSI(i, j)) / hq;
12
13 for i = 2 : Nx - 1
14     for j = 2 : Ny - 1
15         if G(i, j) == -1 && G(i+1, j) > 0
16             ZITA(i, j) = 2 * (PSI(i+1, j) - PSI(i, j)) / hq;
17         elseif G(i, j) == -1 && G(i, j-1) > 0
18             ZITA(i, j) = 2 * (PSI(i, j-1) - PSI(i, j)) / hq;
19         elseif G(i, j) == -1 && G(i, j+1) > 0
20             ZITA(i, j) = 2 * (PSI(i, j+1) - PSI(i, j)) / hq;
21         end
22     end
23 end
24
25 end

```

Code 3 RK4.m. Routine that integrates using the previously discussed Runge-Kutta method.

```

1 function yNew = RK4(t, yOld, RHSfunction, Dt)
2
3 t1 = t;
4 y1 = yOld;
5 f1 = RHSfunction(t1, y1);
6
7 t2 = t + Dt / 2;
8 y2 = yOld + Dt * 0.5 * f1;
9 f2 = RHSfunction(t2, y2);
10
11 t3 = t + Dt / 2;
12 y3 = yOld + Dt * 0.5 * f2;
13 f3 = RHSfunction(t3, y3);
14
15 t4 = t + Dt;
16 y4 = yOld + Dt * f3;
17 f4 = RHSfunction(t4, y4);
18
19 yNew = yOld + Dt * ((f1 + f4) ./ 6 + (f2 + f3) ./ 3);
20
21 end

```


Code 4 ZitaRHS.m. Function of the right-hand side of the ζ partial differential equation.

```

1 function dZdt = ZitaRHS(~, ZITA, Re, h, hq, U, V, G, Nx, Ny)
2
3 dZdt = zeros(Nx, Ny);
4
5 i = 2 : Nx-1;
6 j = 2 : Ny-1;
7 dZdt(i, j) = -0.5 * ( ...
8     U(i+1, j) .* ZITA(i+1, j) - U(i-1, j) .* ZITA(i-1, j) + ...
9     V(i, j+1) .* ZITA(i, j+1) - V(i, j-1) .* ZITA(i, j-1) + ...
10    U(i, j) .* (ZITA(i+1, j) - ZITA(i-1, j)) + ...
11    V(i, j) .* (ZITA(i, j+1) - ZITA(i, j-1)) ...
12    ) / (2 * h) + ( ...
13    ZITA(i+1, j) + ZITA(i-1, j) + ...
14    ZITA(i, j+1) + ZITA(i, j-1) - 4 * ZITA(i, j) ...
15    ) / (Re * hq);
16
17 dZdt(G < 0) = 0;
18
19 end

```

Code 5 PathlinesRHS.m. Function of the right-hand side of a *pathline* partial differential equation.

```

1 function dxdt = PathlinesRHS(~, xP, x, y, U, V, Dx, Dy, Nx, Ny)
2
3 i = floor(xP(1) / Dx) + 1;
4 j = floor(xP(2) / Dy) + 1;
5
6 xNode = [x(min(Nx, i)); y(j)];
7
8 csix = (xP(1) - xNode(1)) / Dx;
9 csiy = (xP(2) - xNode(2)) / Dy;
10
11 if i+1 <= Nx && j+1 <= Ny && i > 0 && j > 0
12     u = [1-csix, csix] * U([i, i+1], [j, j+1]) * [1-csiy; csiy];
13     v = [1-csix, csix] * V([i, i+1], [j, j+1]) * [1-csiy; csiy];
14 else
15     u = 0.05;
16     v = 0;
17 end
18
19 dxdt = [u; v];
20
21 end

```

Code 6 zita2psi.m. Routine that computes the stream function ψ from ζ .

```

1 function [PSI, zita] = zita2psi(zita, ZITA, PSI, Lap, G, Nx, Ny, hq, ...
2     PsiS, PsiN, PsiW, PsiE, PsiCyl)
3
4 zita(G(G > 0)) = ZITA(G > 0);
5
6 zita(G(2:end-1, 2)) = zita(G(2:end-1, 2)) - PsiS(2:end-1) / hq;
7 zita(G(2:end-1, end-1)) = zita(G(2:end-1, end-1)) - PsiN(2:end-1) / hq;
8 zita(G(2, 2:end-1)) = zita(G(2, 2:end-1)) - PsiW(2:end-1)' / hq;

```

```

9 zita(G(end-1, 2:end-1)) = zita(G(end-1, 2:end-1)) - PsiE(2:end-1)' / hq;
10
11 i = 2 : Nx-1;
12 j = 2 : Ny-1;
13 D = G(i, j);
14 zita(D(and(G(i, j) > 0, G(i, j+1) == -1))) = ...
15     zita(D(and(G(i, j) > 0, G(i, j+1) == -1))) ...
16     - PsiCyl / hq;
17 zita(D(and(G(i, j) > 0, G(i, j-1) == -1))) = ...
18     zita(D(and(G(i, j) > 0, G(i, j-1) == -1))) ...
19     - PsiCyl / hq;
20 zita(D(and(G(i, j) > 0, G(i+1, j) == -1))) = ...
21     zita(D(and(G(i, j) > 0, G(i+1, j) == -1))) ...
22     - PsiCyl / hq;
23 zita(D(and(G(i, j) > 0, G(i-1, j) == -1))) = ...
24     zita(D(and(G(i, j) > 0, G(i-1, j) == -1))) ...
25     - PsiCyl / hq;
26
27 psi = Lap \ zita;
28 PSI(G > 0) = psi;
29
30 end

```

Code 7 psi2uv.m. Routine that computes velocity components u and v from the stream function ψ .

```

1 function [U, V] = psi2uv(PSI, U, V, G, h, Nx, Ny, uRef)
2
3 i = 2 : Nx-1;
4 j = 2 : Ny-1;
5 U(i, j) = (PSI(i, j+1) - PSI(i, j-1)) / (2 * h);
6 V(i, j) = -(PSI(i+1, j) - PSI(i-1, j)) / (2 * h);
7
8 U(G < 0) = 0;
9 V(G < 0) = 0;
10 U(1, :) = uRef;
11 U(end, :) = uRef;
12
13 end

```

Code 8 isInsideNACA2418.m. Routine that determines whether or not a given position $x = (x, y)$ lies inside a NACA2418 airfoil.

```

1 function isInside = isInsideNACA2418(x, y, c, le, alpha_deg)
2
3 isInside = false;
4
5 m = 0.02;
6 p = 0.40;
7 t = 0.18 * c;
8
9 theta = @(xc) ...
10     atan(2*m / p^2 * (p - xc) * (xc <= p && xc >= 0) + ...
11     2*m / (1 - p^2) * (p - xc) * (xc > p && xc <= 1));
12 yt = @(xc) ...
13     5 * t * (0.2969 * sqrt(xc) - 0.1260 * xc - 0.3516 * xc.^2 + ...

```

```

14     0.2843 * xc.^3 - 0.1015 * xc.^4);
15 yc = @(xc) ...
16     m / p^2 * (2 * p * xc - xc.^2) * (xc >= 0 && xc <= p) + ...
17     m / (1 - p)^2 * (1 - 2*p + 2*p*xc - xc.^2) * (xc > p && xc <= 1);
18
19 xU = @(xc) c * (xc - yt(xc) * sin(theta(xc)));
20 xL = @(xc) c * (xc + yt(xc) * sin(theta(xc)));
21
22 yU = @(xc) c * (yc(xc) + yt(xc) .* cos(theta(xc)));
23 yL = @(xc) c * (yc(xc) - yt(xc) .* cos(theta(xc)));
24
25 Upper = @(xc) Rotate([xU(xc), yU(xc)], alpha_deg) + le(:);
26 Lower = @(xc) Rotate([xL(xc), yL(xc)], alpha_deg) + le(:);
27
28 yUpper = @(xc) [0, 1] * Upper(xc);
29 yLower = @(xc) [0, 1] * Lower(xc);
30
31 xte = le(1) + c * cosd(alpha_deg);
32
33 if x >= le(1) && x <= xte
34     if y >= yLower((x - le(1)) / (c * cosd(alpha_deg))) && y <= yUpper((x -
35         le(1)) / (c * cosd(alpha_deg)))
36         isInside = true;
37     end
38 end
39 end

```

4 Results

Results of each simulation are shown here. It is worth pointing out that black regions in the plot are only graphical representations of the obstacles and that the actual computed obstacles are the ones previously shown in figures 2 and 3.

Figure 4 shows the time history of the total enstrophy as previously defined in equation (46).

Wake flow past the circular cylinder is shown in figures 5 to 12, while wake flow past the airfoil is shown in figures 13 to 26, for two different angles of attack. Resulted

diagrams have been captured at different time instants.

It is worth noticing the qualitative differences between the two geometries. As expected, the wake flow past the airfoil is significantly less intense than the one past the circular cylinder, even when the airfoil is near stall. In fact, the wake flow past the airfoil quickly mitigates during the beginning of the simulation.

On the other hand, the cylinder simulation allows to easily appreciate a Kármán vortex street and a significant wake turbulence.

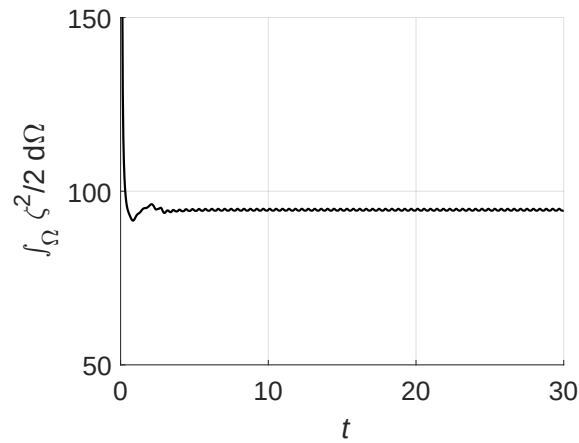


Figure 4 Time history of the total enstrophy. Other than a significant overshoot error during the beginning of the simulation, the total enstrophy is approximately constant as required and expected through the entire simulation.

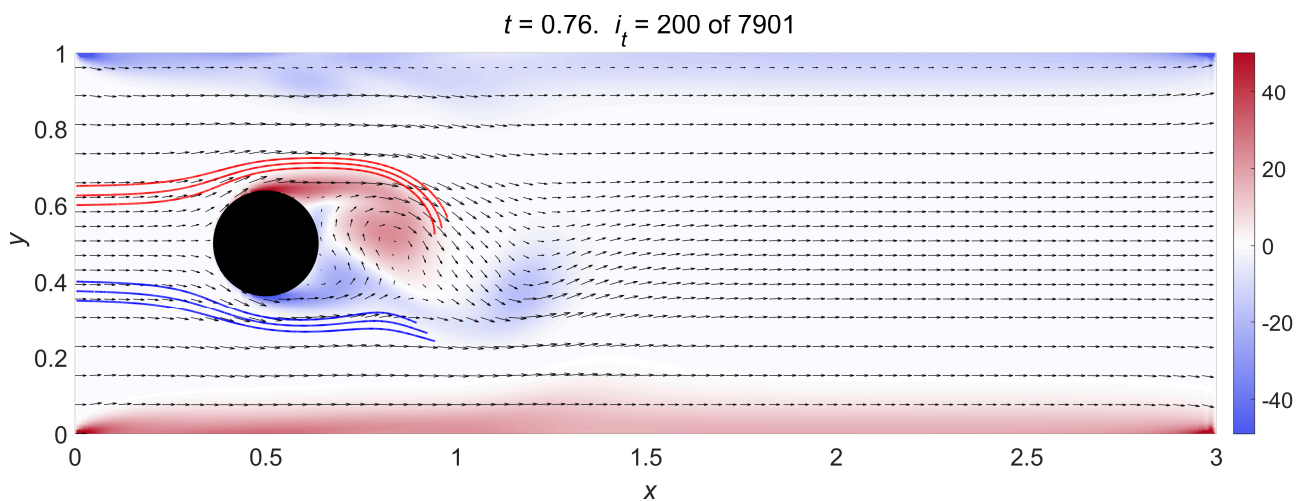


Figure 5 Wake flow past a circular cylinder at $t = 0.76$. Heavy red and blue lines are *streaklines* and the color plot refers to ζ values over the domain.

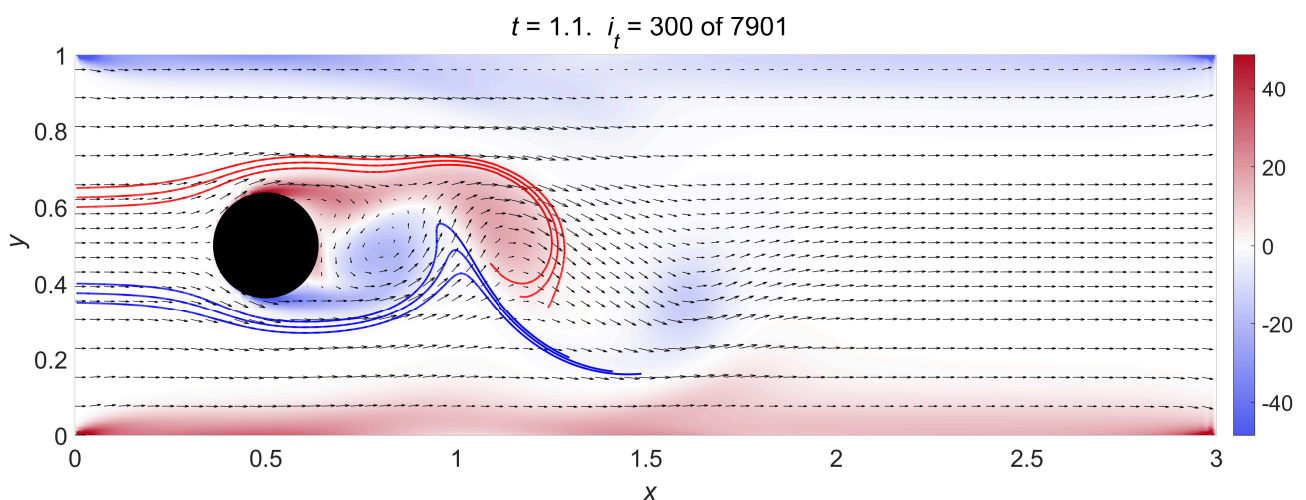


Figure 6 Wake flow past a circular cylinder at $t = 1.1$. Heavy red and blue lines are *streaklines* and the color plot refers to ζ values over the domain.

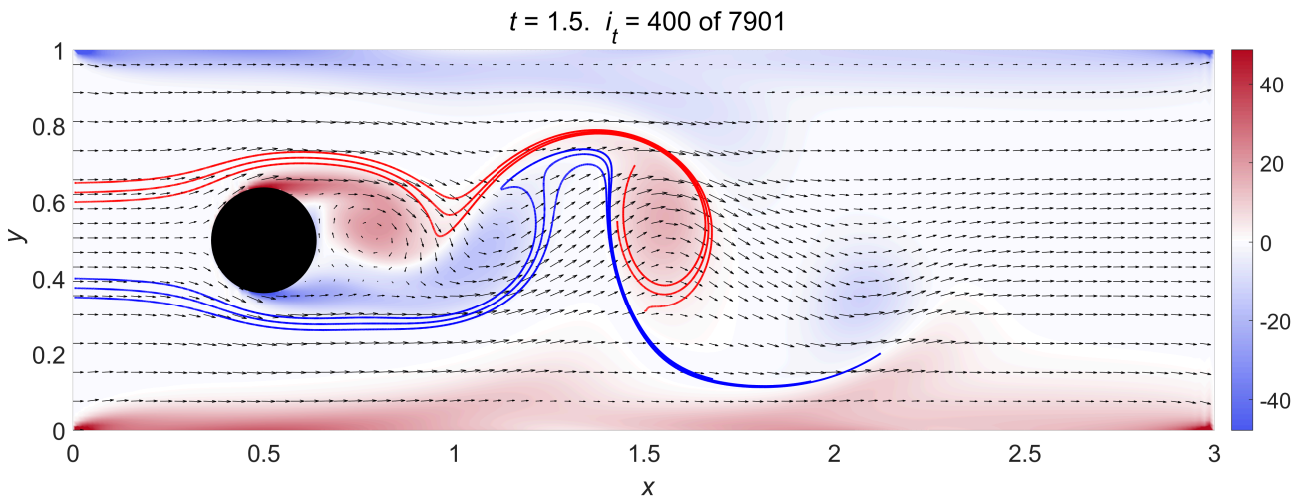


Figure 7 Wake flow past a circular cylinder at $t = 1.5$. Heavy red and blue lines are *streaklines* and the color plot refers to ζ values over the domain.

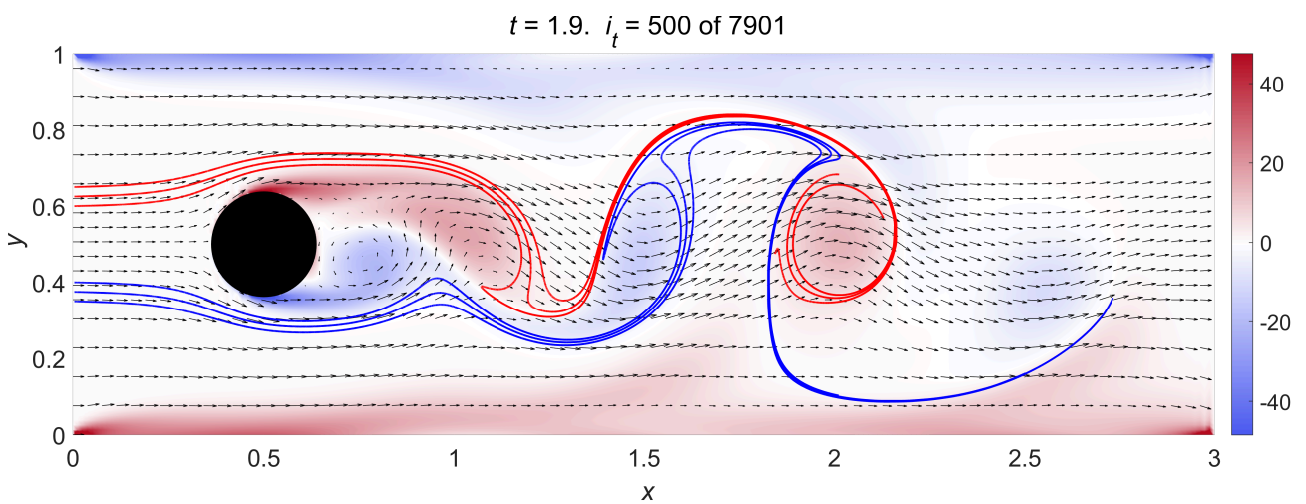


Figure 8 Wake flow past a circular cylinder at $t = 1.9$. Heavy red and blue lines are *streaklines* and the color plot refers to ζ values over the domain.

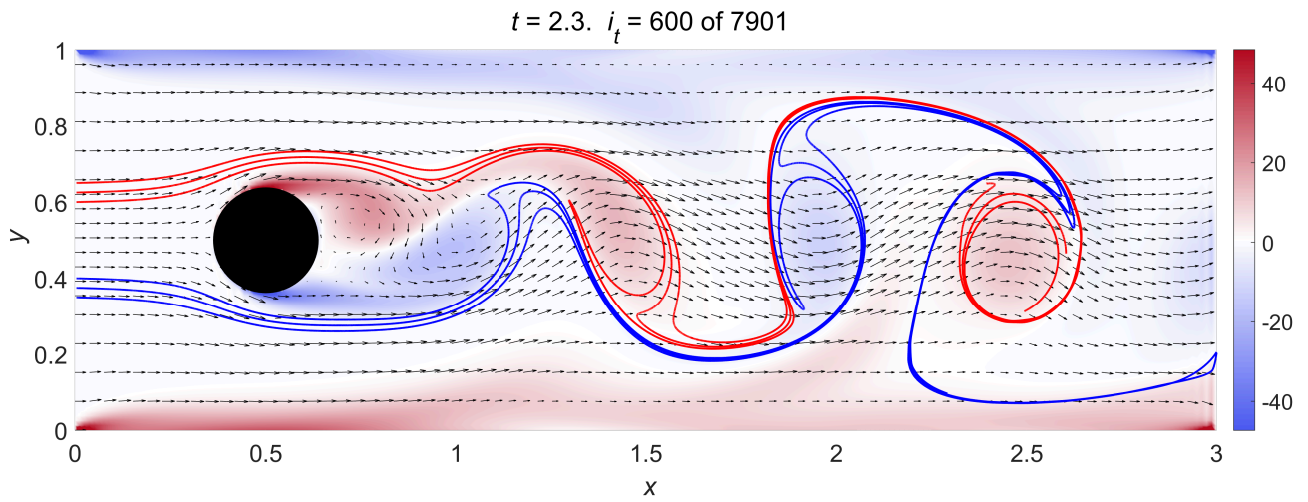


Figure 9 Wake flow past a circular cylinder at $t = 2.3$. Heavy red and blue lines are *streaklines* and the color plot refers to ζ values over the domain.

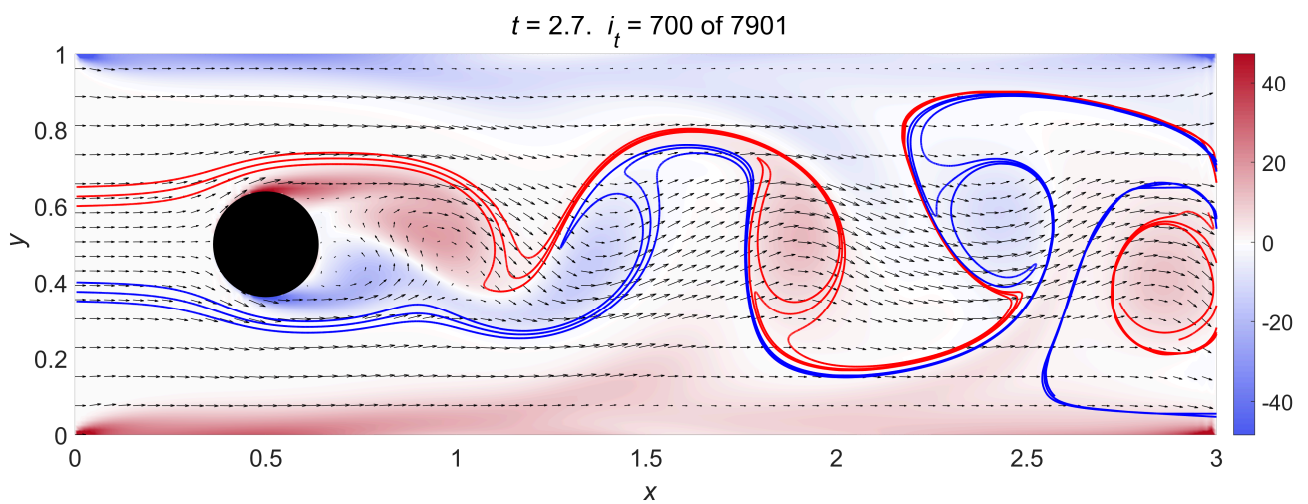


Figure 10 Wake flow past a circular cylinder at $t = 2.7$. Heavy red and blue lines are *streaklines* and the color plot refers to ζ values over the domain.

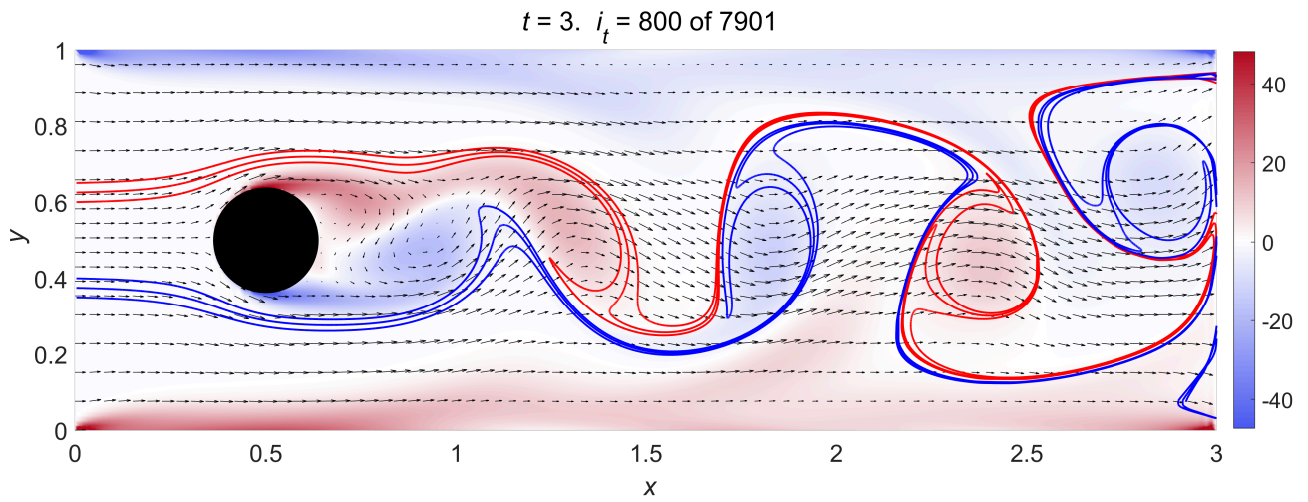


Figure 11 Wake flow past a circular cylinder at $t = 3.0$. Heavy red and blue lines are *streaklines* and the color plot refers to ζ values over the domain.

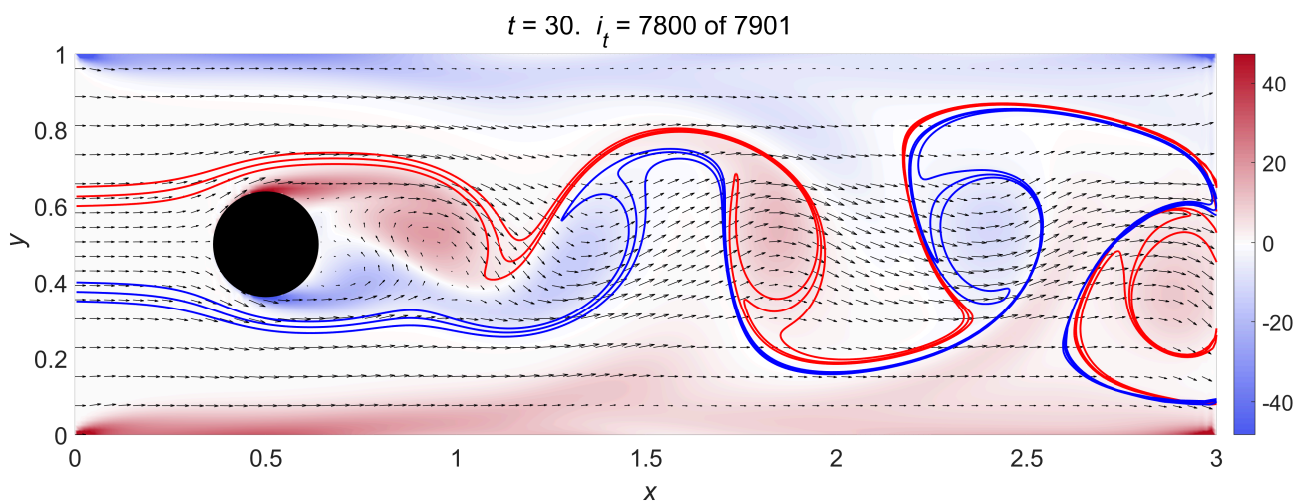


Figure 12 Wake flow past a circular cylinder at $t = T = 30$. Heavy red and blue lines are *streaklines* and the color plot refers to ζ values over the domain.

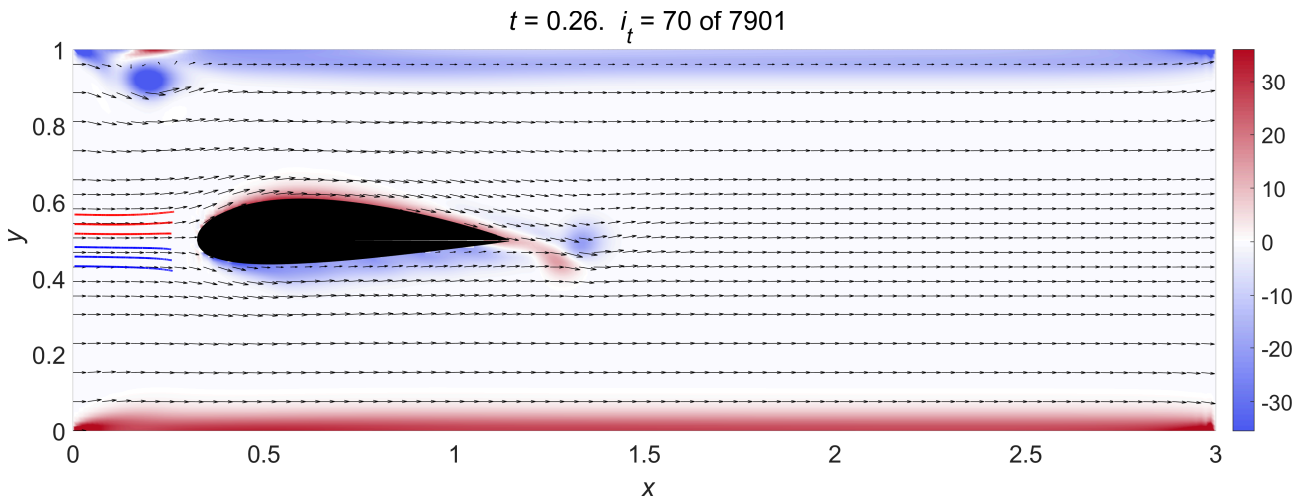


Figure 13 Wake flow past a 2418 NACA airfoil at angle of attack $\alpha = 0$ deg, $t = 0.26$. Heavy red and blue lines are *streaklines* and the color plot refers to ζ values over the domain.

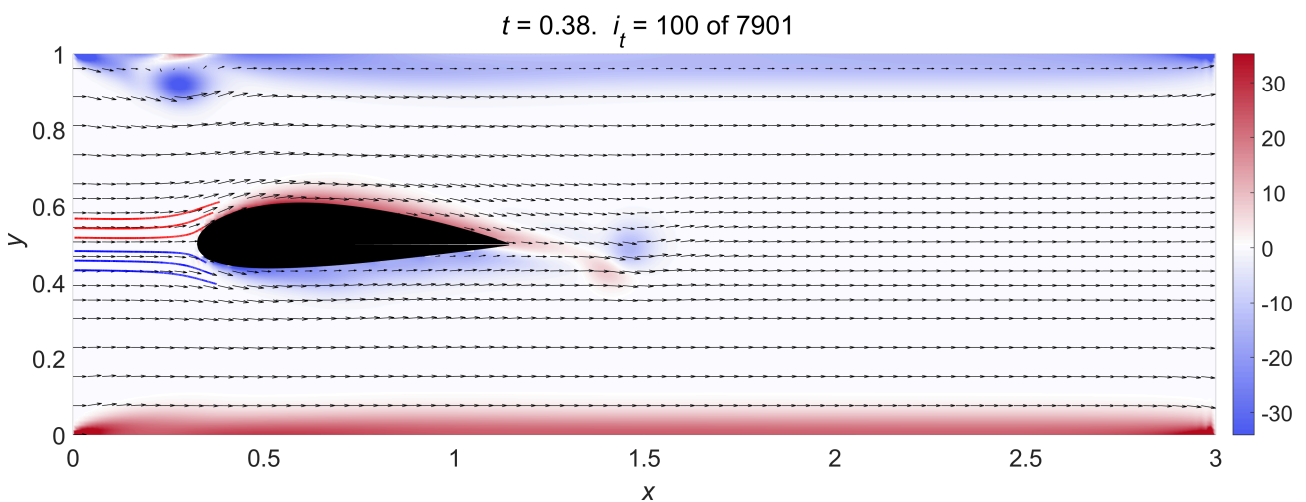


Figure 14 Wake flow past a 2418 NACA airfoil at angle of attack $\alpha = 0$ deg, $t = 0.38$. Heavy red and blue lines are *streaklines* and the color plot refers to ζ values over the domain.

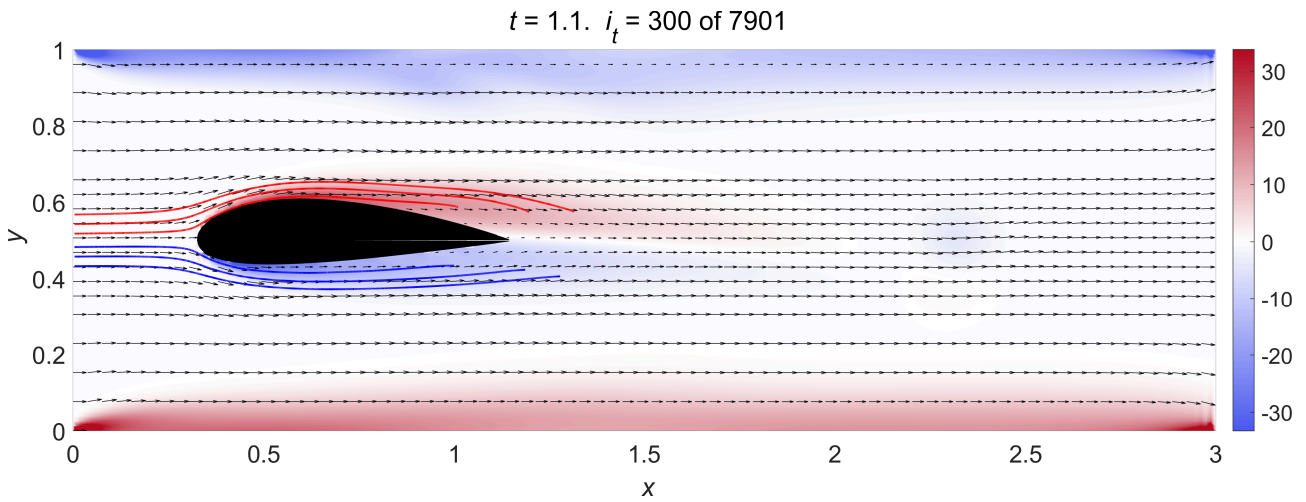


Figure 15 Wake flow past a 2418 NACA airfoil at angle of attack $\alpha = 0$ deg, $t = 1.1$. Heavy red and blue lines are *streaklines* and the color plot refers to ζ values over the domain.

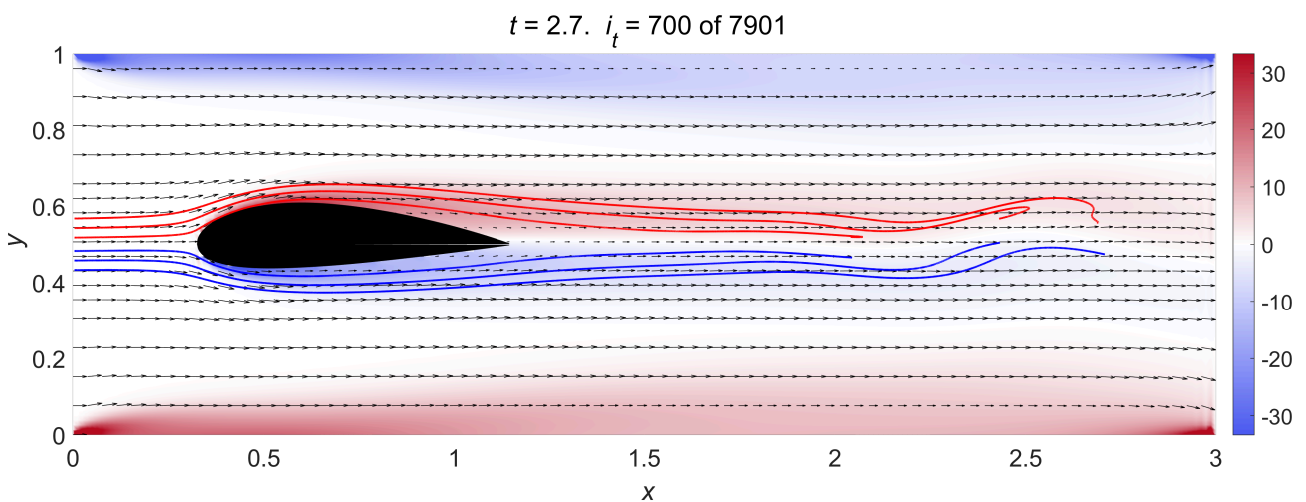


Figure 16 Wake flow past a 2418 NACA airfoil at angle of attack $\alpha = 0$ deg, $t = 2.7$. Heavy red and blue lines are *streaklines* and the color plot refers to ζ values over the domain.

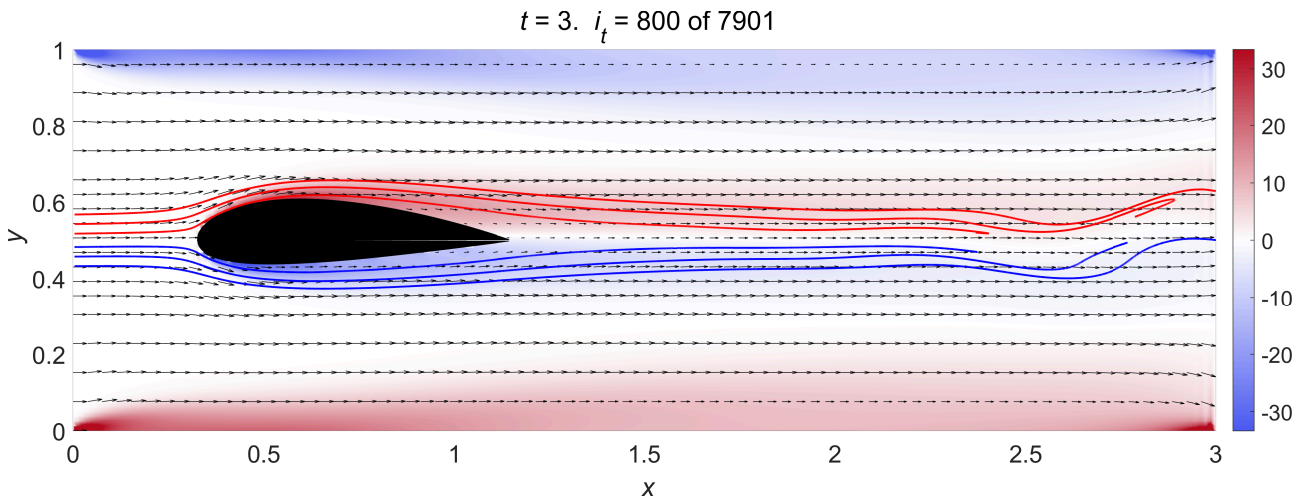


Figure 17 Wake flow past a 2418 NACA airfoil at angle of attack $\alpha = 0^\circ$, $t = 3.0$. Heavy red and blue lines are *streaklines* and the color plot refers to ζ values over the domain.

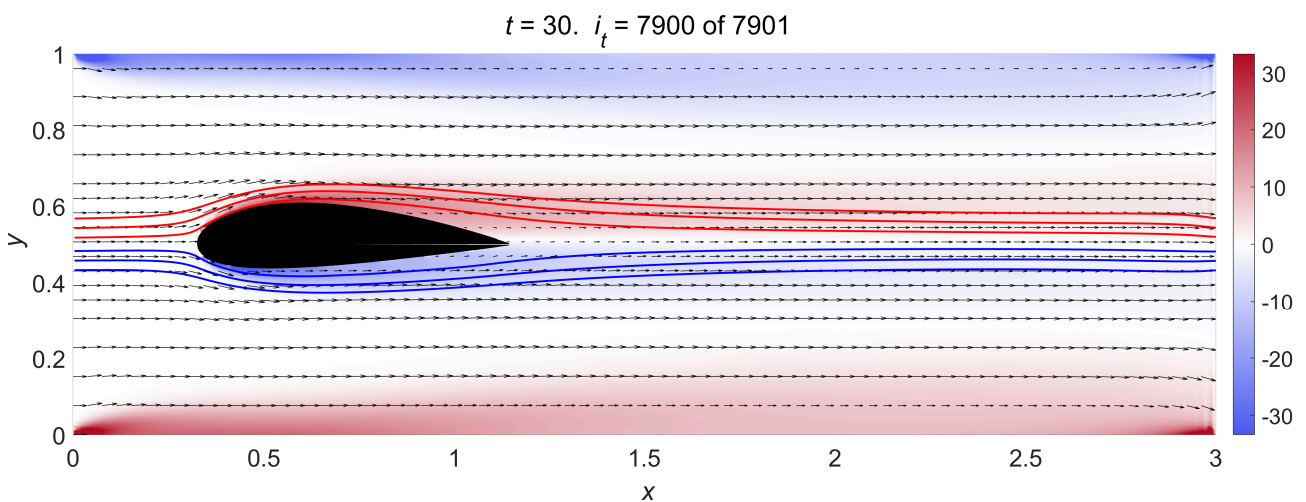


Figure 18 Wake flow past a 2418 NACA airfoil at angle of attack $\alpha = 0^\circ$, $t = T = 30.0$. Heavy red and blue lines are *streaklines* and the color plot refers to ζ values over the domain.

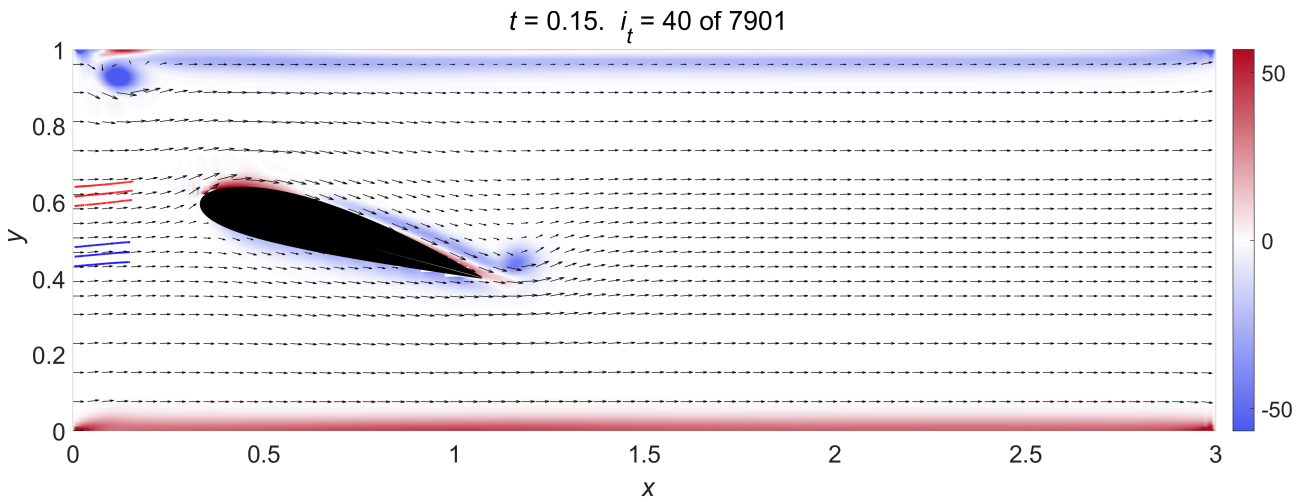


Figure 19 Wake flow past a 2418 NACA airfoil at angle of attack $\alpha = 15$ deg, $t = 0.15$. Heavy red and blue lines are *streaklines* and the color plot refers to ζ values over the domain.

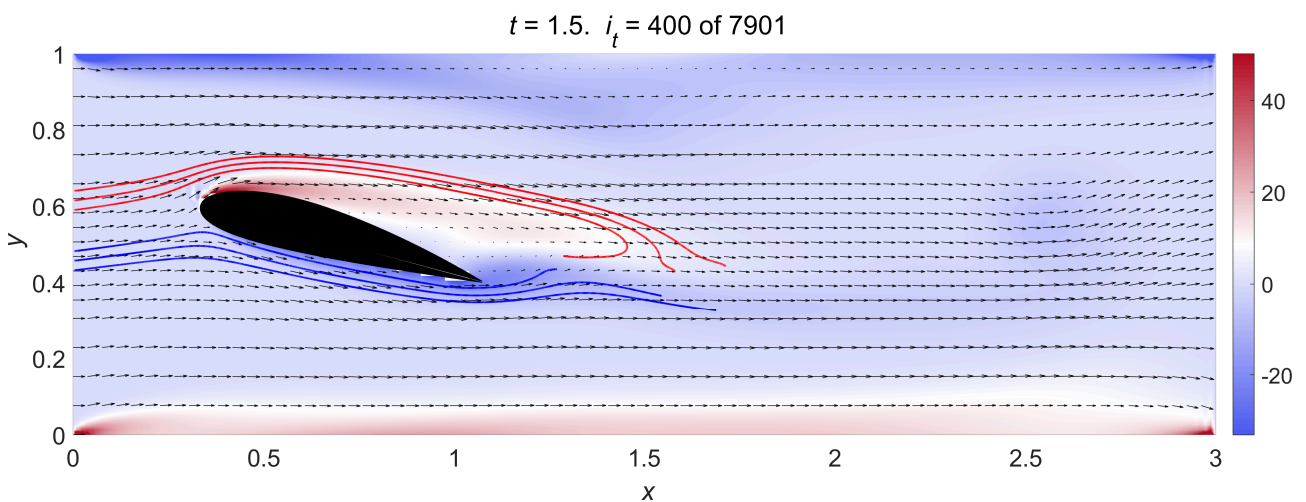


Figure 20 Wake flow past a 2418 NACA airfoil at angle of attack $\alpha = 15$ deg, $t = 1.5$. Heavy red and blue lines are *streaklines* and the color plot refers to ζ values over the domain.

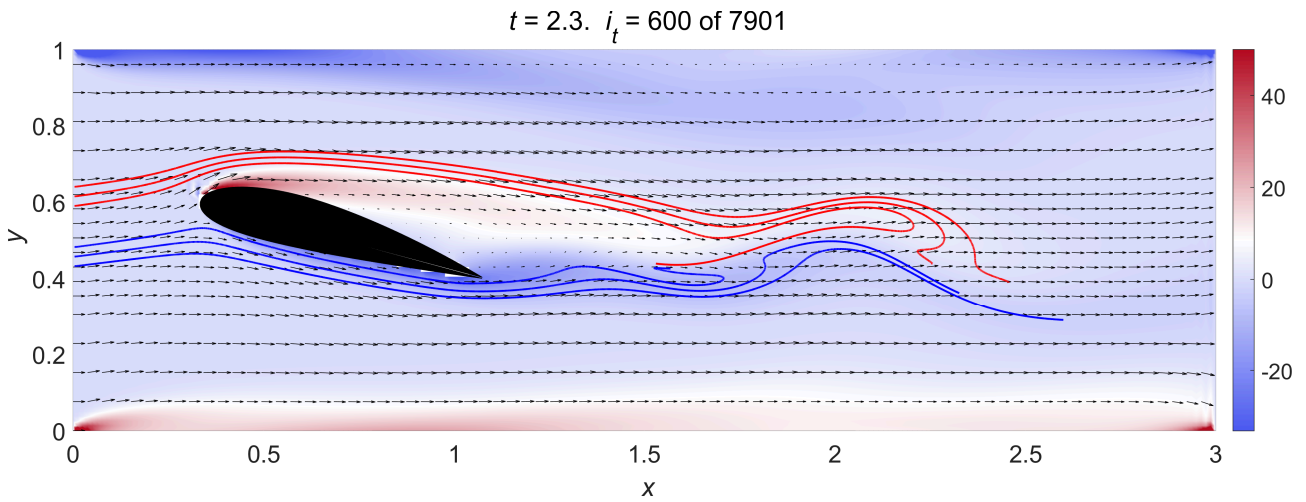


Figure 21 Wake flow past a 2418 NACA airfoil at angle of attack $\alpha = 15$ deg, $t = 2.3$. Heavy red and blue lines are *streaklines* and the color plot refers to ζ values over the domain.

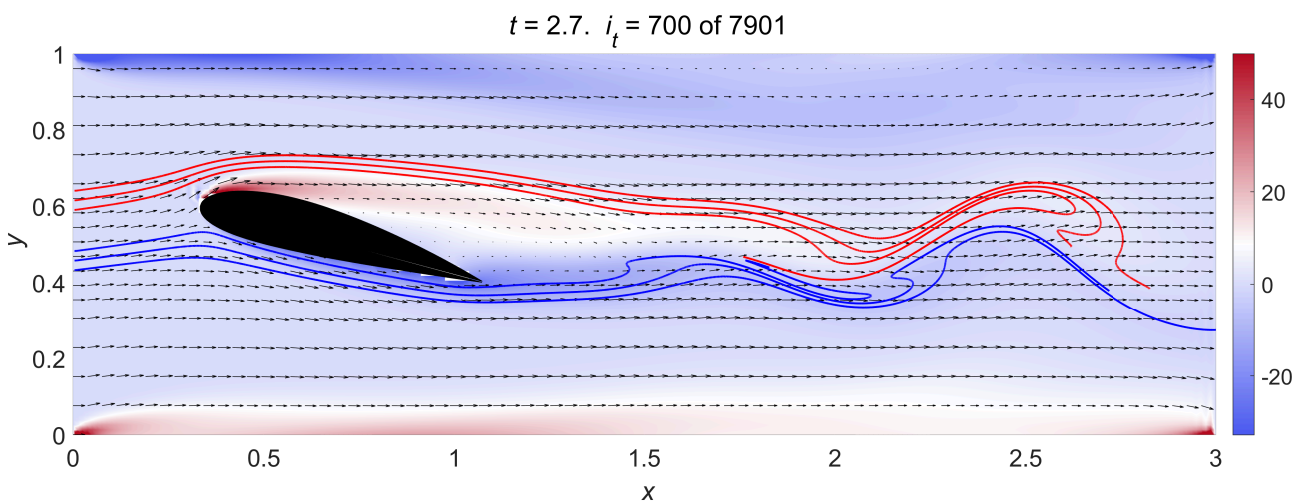


Figure 22 Wake flow past a 2418 NACA airfoil at angle of attack $\alpha = 15$ deg, $t = 2.7$. Heavy red and blue lines are *streaklines* and the color plot refers to ζ values over the domain.

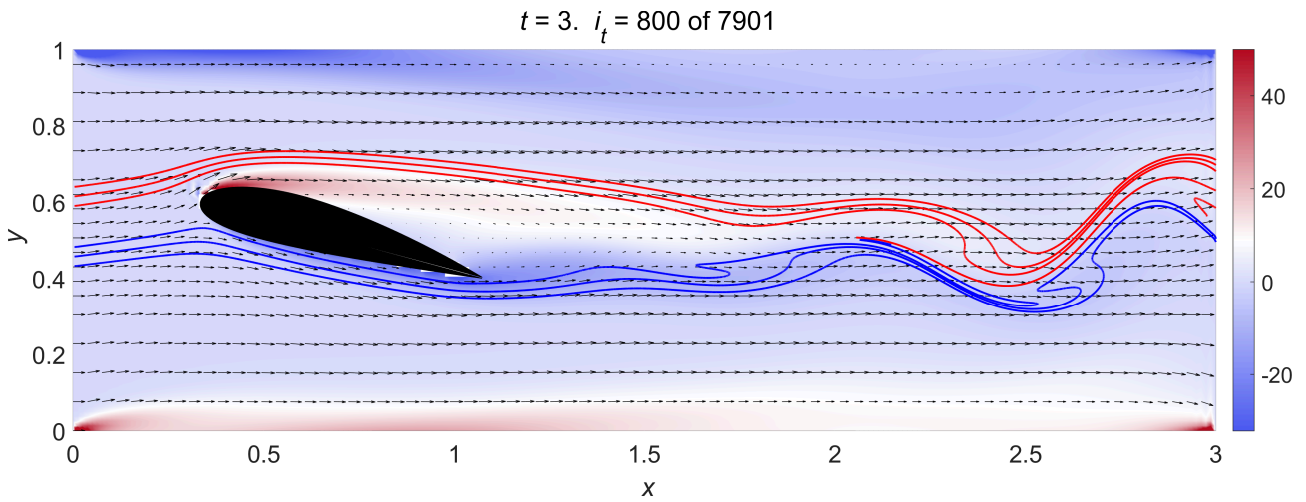


Figure 23 Wake flow past a 2418 NACA airfoil at angle of attack $\alpha = 15$ deg, $t = 3.0$. Heavy red and blue lines are *streaklines* and the color plot refers to ζ values over the domain.

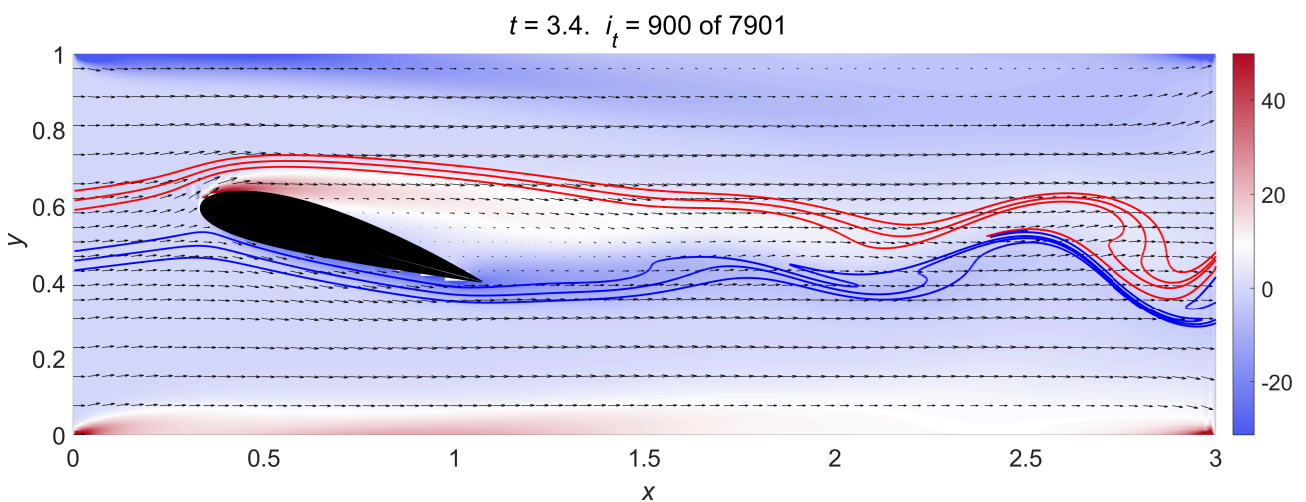


Figure 24 Wake flow past a 2418 NACA airfoil at angle of attack $\alpha = 15$ deg, $t = 3.4$. Heavy red and blue lines are *streaklines* and the color plot refers to ζ values over the domain.

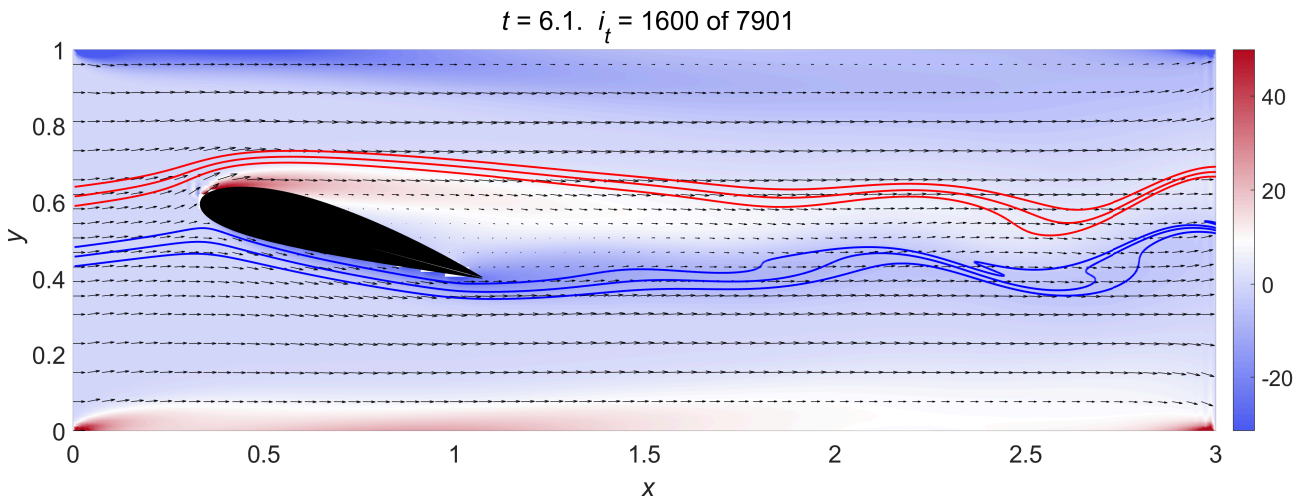


Figure 25 Wake flow past a 2418 NACA airfoil at angle of attack $\alpha = 15$ deg, $t = 6.1$. Heavy red and blue lines are *streaklines* and the color plot refers to ζ values over the domain.

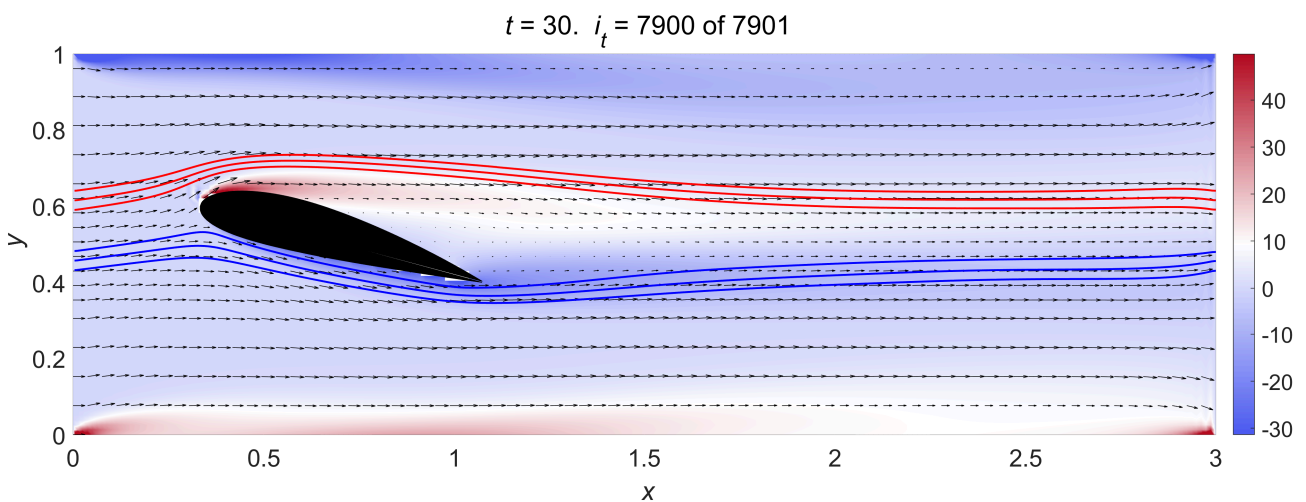


Figure 26 Wake flow past a 2418 NACA airfoil at angle of attack $\alpha = 15$ deg, $t = T = 30$. Heavy red and blue lines are *streaklines* and the color plot refers to ζ values over the domain.

References

- [1] J. Anderson. *Fundamentals of aerodynamics*. McGraw-Hill, 2011.
- [2] Milton Van Dyke. *An Album of Fluid Motion*. The Parabolic Press, 1988.
- [3] Ira H. Abbott and Albert E. Von Doenhoff. *Theory of Wing Sections*. Dover Publications, 1949.
- [4] Rodolfo Monti and Raffaele Savino. *Aerodinamica*. Liguori Editore, 1998.
- [5] John C. Tannehill, Dale A. Anderson, and Richard H. Pletcher. *Computational Fluid Mechanics and Heat Transfer*. Taylor & Francis, 1984.
- [6] Arieh Iserles. *A First Course in the Numerical Analysis of Differential Equations*. Cambridge University Press, 2009.